LeetCode HOT 100 · 可视复习卡全集（121 题）

每题一卡：题面 + 示例 + 核心套路可视化 + 一句话口诀 · 2026-09-27

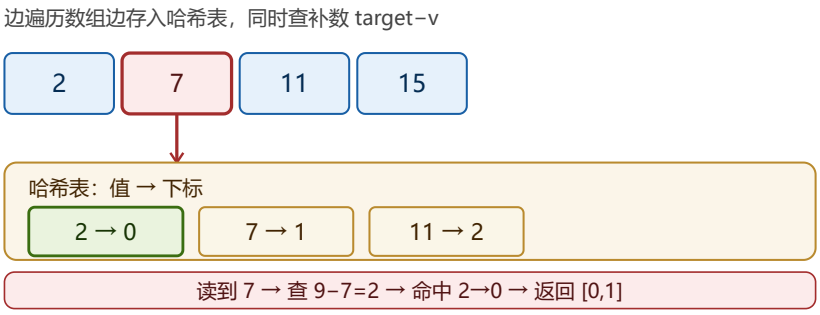
哈希

#1 两数之和 简单 · 数组、哈希表

给定整数数组 nums 和目标值 target，找出和为目标值的两个整数，返回它们的数组下标。

```
输入：nums = [2,7,11,15], target = 9
输出：[0,1]
```

⚡ 哈希补数：边遍历边查 target−当前值 是否已入表，命中即返回下标。

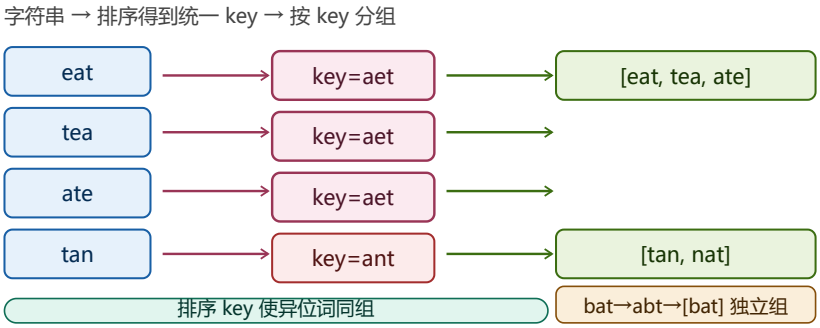


#49 字母异位词分组 中等 · 哈希表 / 字符串 / 排序

把字母异位词分到同一组（字母相同、顺序不同）。

```
strs =
["eat","tea","tan","ate","nat","bat"]
→ [["bat"],["nat","tan"],
["ate","eat","tea"]]
```

⚡ 排序后作为 key：eat→aet、tea→aet 归一组；哈希表 map[key] = 列表。

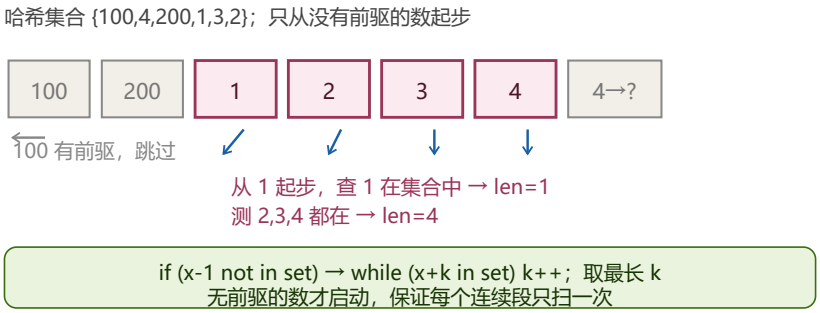


#128 最长连续序列 中等 · 并查集 / 数组 / 哈希表

未排序数组里找数字连续的最长序列（无需原数组连续）。

```
nums=[100,4,200,1,3,2] → 4（1,2,3,4）
```

⚡ 哈希集去重；只从「无前驱」的数开始向后数，O(n)。



双指针

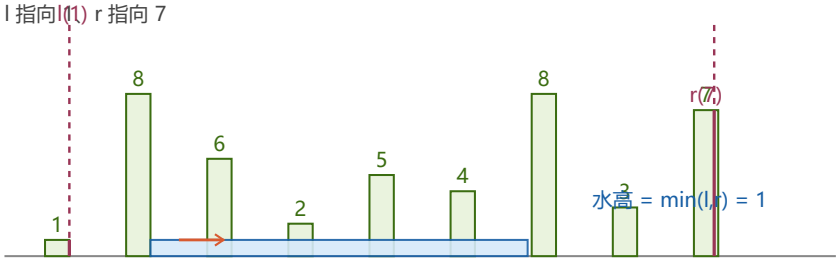
#11 盛最多水的容器 中等 · 贪心 / 数组 / 双指针

选两条线构成容器，求能装水的最大面积。

```
height=[1,8,6,2,5,4,8,3,7] → 49
```

⚡ 双指针从两端向中间收，每次移动「较短的那根」；面积=短边×距离。

柱高按数值等比（1格=8px）
l 指向l(1) r 指向 7



面积 = min(1,7)×9 = 9；l 更短 → l 右移找更高的边，直到相遇

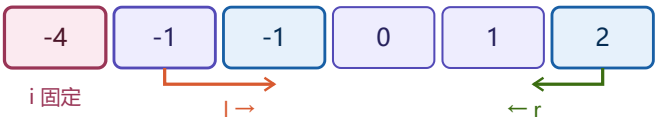
#15 三数之和 中等 · 数组、双指针、排序

在数组中找出所有和为 0 且不重复的三元组 [nums[i], nums[j], nums[k]]。

```
输入：nums = [-1,0,1,2,-1,-4]
输出：[[-1,-1,2],[-1,0,1]]
```

⚡ 排序 + 固定 i + 双指针 l/r 夹逼：和大了 r 左移、小了 l 右移，跳过重复值去重。

排序后固定 i，l 向右、r 向左，向中间夹逼



sum = nums[i] + nums[l] + nums[r]

=0 记录并 l++、r--；<0 → l++；>0 → r--（跳过重复值）

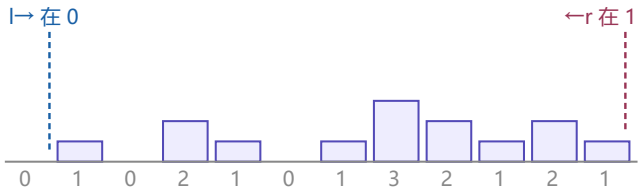
#42 接雨水 困难 · 双指针 / 单调栈 / 动态规划

柱状图中能接住多少雨水（宽度 1）。

```
height = [0,1,0,2,1,0,1,3,2,1,2,1]
→ 6
```

⚡ 双指针：矮的一侧决定水位，谁矮谁动；ans += 矮侧max - 当前高度（若为正）。

l=0 在左，r=11 在右：谁矮谁动，低侧水位即接水量



双指针：矮侧决定水位，谁矮谁动
ans += 矮侧max - h[i]；例：两端低洼各积 1，共 6

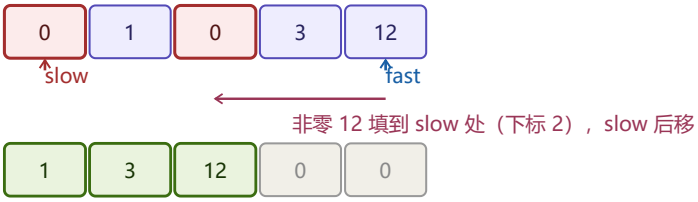
#283 移动零 简单 · 数组、双指针

把所有 0 移到数组末尾，保持非零元素相对顺序（原地）。

```
输入：nums = [0,1,0,3,12]
输出：[1,3,12,0,0]
```

⚡ 快慢指针：slow 指向下一个非零应放位置，fast 找非零；非零值填前，末尾补 0。

slow 指向待写入位置，fast 扫描非零元素



扫描完后，slow 之后全部置 0
等价写法：遇非零与 slow 处交换 → 0 自动沉到尾部

滑动窗口

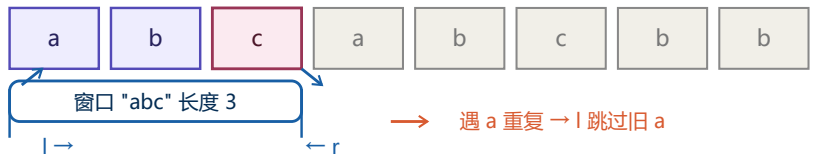
#3 无重复字符的最长子串 中等 · 哈希表 / 字符串 / 滑动窗口

找出不含重复字符的「最长子串」的长度。

```
s = "abcabcbb" → 3
最长子串 "abc"
```

⚡ 滑动窗口：右指针扩张，遇到重复就收缩左指针，窗口永远无重复。

滑动窗口 [l, r]，哈希表记字符最近下标



窗口 [l, r] 恒无重复；哈希表存字符→下标
遇重复：l = max(l, 旧下标+1)；每步更新 maxLen

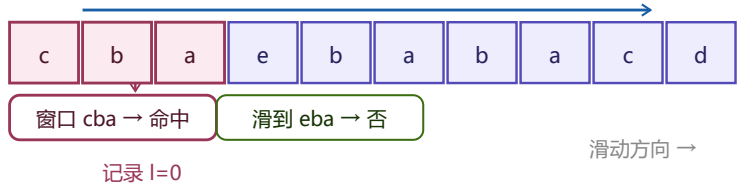
#438 找到字符串中所有字母异位词 中等 · 哈希表 / 字符串 / 滑动窗口

找 s 中所有 p 的异位词（乱序字母串）起始下标。

```
s="cbaebabacd", p="abc" → [0,6]
```

⚡ 定长滑动窗口：窗口字符计数与 p 完全一致时记录 l。

p=abc（长度3）；窗口滑一步 → 进一个出一个



计数数组 window 与 p 对比；全 0 差 → 记录下标
窗口长度固定 = |p|，滑到末尾即可，O(n)

子串

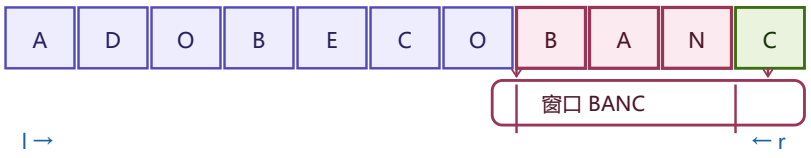
#76 最小覆盖子串 困难 · 哈希表 / 字符串 / 滑动窗口

找 s 中最短子串，包含 t 的所有字符（含重复）。

```
s="ADOBECODEBANC", t="ABC"
→ "BANC"
```

⚡ 滑动窗口：右扩到「全包含」→ 左缩找最短，用 need/satisfied 计数。

t={A:1,B:1,C:1}；窗口右扩达标后左缩



A/B/C 齐全 → 尝试 l 左缩，缺字符才再右扩
满足集合数 = len(t) 时贡献一个候选，取窗口最短的一条

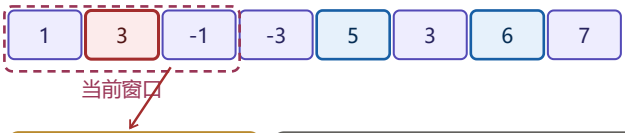
#239 滑动窗口最大值 困难 · 单调队列、滑动窗口

每个大小为 k 的窗口求最大值，窗口每次右移一位。

```
输入：nums = [1,3,-1,-3,5,3,6,7], k = 3
输出：[3,3,5,5,6,7]
```

⚡ 单调递减队列：新元素入队时弹出队尾所有更小值；队头永远最大，过期下标从队头弹出。

窗口 [1,3,-1] 最大值 3 → 单调队列维护递减



单调队列：存下标
3 在下，-1 在上

入 5：弹出队尾 -1、3（都小于 5）→ 队 [5]
窗口滑出旧下标则从队头弹；每次答案 = 队头值

出队两种：① 队尾弹小于新值的；② 队头弹已滑出窗口的
队头即当前窗口最大值，答案依次 [3,3,5,5,6,7]

#560 和为 K 的子数组 中等 · 数组、哈希表、前缀和

统计和为 k 的连续子数组个数。

```
输入：nums = [1,1,1], k = 2
输出：2
```

⚡ 前缀和 + 哈希：每算一个前缀和 p，查 p−k 出现次数累加；前缀和计数入表。

pre[i] = 前 i 项和；子数组 [j+1..i] 的和 = pre[i] − pre[j]



pre: 0, 1, 2, 3　　查 pre−k: 0−2, 1−2, 2−2, 3−2

pre=2 → 查 0（有1次）→ 子数组 [1,1] ✓；pre=3 → 查 1（有1次）→ [1,1] ✓

哈希表 {0:1} 起步；每步先查再登记当前 pre
累计 2 个和为 2 的子数组 ✓ O(n)

普通数组

#41 缺失的第一个正数 困难 · 数组、哈希表

在未排序数组中找到没出现的最小正整数，要求 O(1) 额外空间。

```
输入：nums = [1,2,0]
输出：3
解释：范围 [1,2] 中的数字都在数组中
```

⚡ 原地哈希：值 x 放到下标 x−1；扫完后第一个「下标+1≠值」处，下标+1 即答案。

把 1 放进下标 0，把 2 放进下标 1…… 值 x → 下标 x−1

原始 nums：



归位后（值 1→下标0，值 2→下标1）：



扫到下标 2：nums[2]=0 ≠ 3 → 缺失的最小正整数 = 3

#53 最大子数组和 中等 · 数组、分治、动态规划

找和最大的连续子数组，返回最大和。

```
输入：nums = [-2,1,-3,4,-1,2,1,-5,4]
输出：6
解释：[4,-1,2,1] 的和最大，为 6
```

⚡ Kadane：cur = max(当前值, cur+当前值)；ans = max(ans, cur)。前缀和变负数就丢弃重启。

逐步递推 cur：要么续上前段，要么从当前值重新开始



cur: -2, 1, -2, 4, 3, 5, 6（红色 = 最优子段起点）

到 4 时前段和 -3 < 0 → 丢弃，cur 重启为 4

子段 [4,-1,2,1] 和 = 6 → 最大

#56 合并区间 中等 · 数组 / 排序

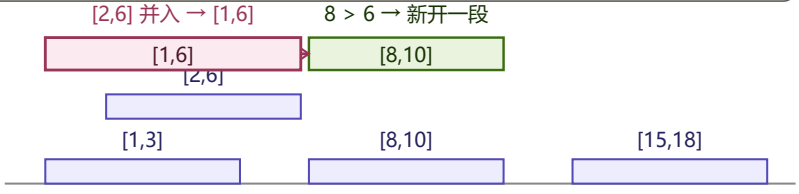
合并所有重叠区间，返回不重叠的覆盖区间。

```
intervals=[[1,3],[2,6],[8,10],[15,18]] →
[[1,6],[8,10],[15,18]]
```

⚡ 按起点排序，依次合并：当前 start ≤ 已合并区间的 end 则取 max(end)。

按 start 排序：重叠则合并，不重叠则新开一段

排序 O(nlogn) → 一趟合并 O(n)：cur.start ≤ last.end 就 max 扩展

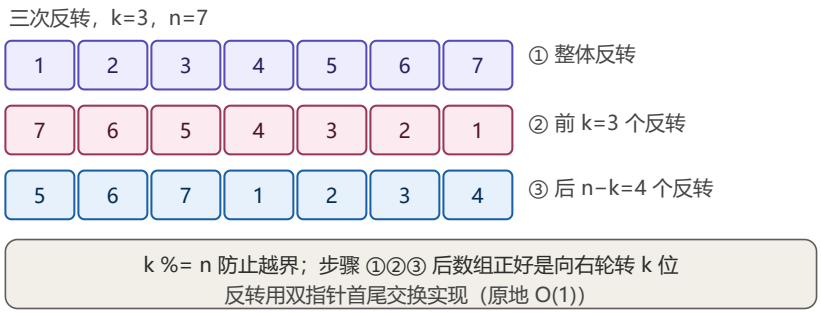


#189 轮转数组 中等 · 数组、数学、双指针

把数组向右轮转 k 个位置（原地，O(1) 额外空间）。

```
输入：nums = [1,2,3,4,5,6,7], k = 3
输出：[5,6,7,1,2,3,4]
```

⚡ 三次反转：整体反转 → 前 k 个反转 → 后 n−k 个反转，即可得到轮转结果。



#238 除自身以外数组的乘积 中等 · 数组、前缀和

answer[i] = 除 nums[i] 外所有元素的乘积，不能用除法。

```
输入：nums = [1,2,3,4]
输出：[24,12,8,6]
```

⚡ 前缀积 × 后缀积：第一遍从左累乘记前缀，第二遍从右累乘记后缀。

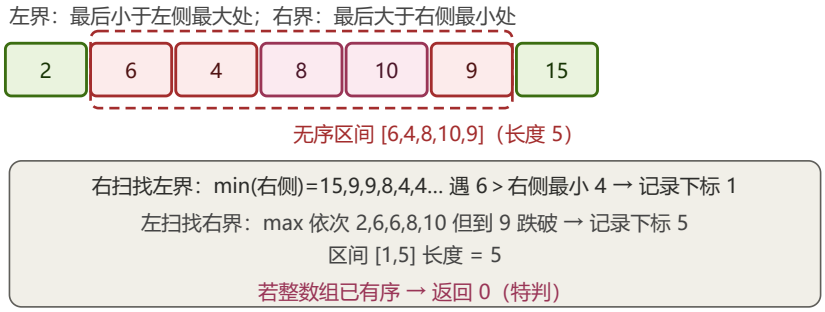


#581 最短无序连续子数组 中等 · 数组、双指针、贪心

找出最短子数组，对其升序排序后整个数组升序。

```
输入：nums = [2,6,4,8,10,9,15]
输出：5
解释：需对 [6,4,8,10,9] 排序
```

⚡ 从左扫找最后一个「大于左侧最大」的乱序点，从右扫找第一个「小于右侧最小」处，二者夹住的区间即答案。



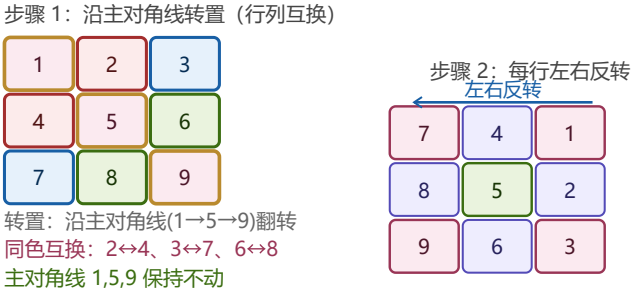
矩阵

#48 旋转图像 中等 · 数组、数学、矩阵

将 n×n 矩阵顺时针旋转 90 度，要求原地。

```
输入：matrix = [[1,2,3],[4,5,6],[7,8,9]]
输出：[[7,4,1],[8,5,2],[9,6,3]]
```

⚡ 先转置（沿主对角线翻转）再左右镜像反转每行，两步完成顺时针 90°。



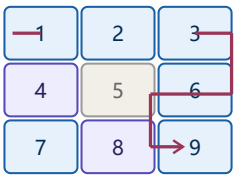
#54 螺旋矩阵 中等 · 数组、矩阵、模拟

按顺时针螺旋顺序返回矩阵所有元素。

```
输入：matrix = [[1,2,3],[4,5,6],[7,8,9]]
输出：[1,2,3,6,9,8,7,4,5]
```

⚡ 四边界收缩：按 上→右→下→左 依次走完一整圈，每走完一条边就收缩该边界。

四个边界：上 top / 下 bottom / 左 left / 右 right 逐圈收缩



依次输出：

1,2,3,6,9,8,7,4,5

边界：top→1 后 top+1；right→6 后 right−1；bottom→7

while left<=right && top<=bottom 一直螺旋走
只剩一行/一列时防止重复打印（条件判断跳过已收缩边）

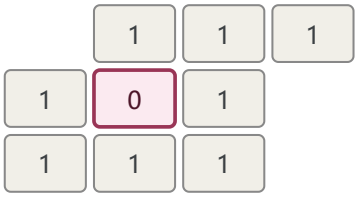
#73 矩阵置零 中等 · 数组、哈希表、矩阵

矩阵中某格为 0，则其整行整列都置 0，要求 O(1) 空间。

```
输入：matrix = [[1,1,1],[1,0,1],[1,1,1]]
输出：[[1,0,1],[0,0,0],[1,0,1]]
```

⚡ 用首行首列当"标记位"：遇到 0 记到第一行/第一列，第二遍据标记清整行整列。

0 出现在 [1][1] → 记首行 row[1]、首列 col[1]



遍历时把 0 信息记到首行首列

col[1] 标记　　row[1] 标记

结果：第1行和第1列全变 0（[1][1] 本身为 0 由 row/col 标记共同清零）

#85 最大矩形 困难 · 单调栈 / 动态规划 / 矩阵

二进制矩阵中找出只含 1 的最大矩形面积。

```
matrix = [["1","0","1","0","0"],
          ["1","0","1","1","1"],
          ["1","1","1","1","1"],
          ["1","0","0","1","0"]]
→ 6
```

⚡ 逐行累加连续 1 的高度 → 每行当柱状图跑 #84 单调栈求最大面积。

每行算出「该格往上的连续 1 高度」，转柱状图



第 3 行高 [3,1,3,2,2]

高度=连续1的个数



中间 3 根公共高 2 → 宽3 × 高2 = 6

每行维护高度数组，用单调栈找每个柱子左右第一个更矮处 → 最大矩形

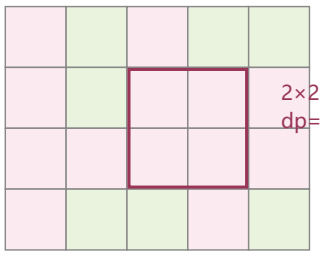
#221 最大正方形 中等 · 数组 / DP / 矩阵

在 0/1 矩阵里找只含 1 的最大正方形，返回面积。

```
[[1,0,1,0,0],[1,0,1,1,1],[1,1,1,1,1],
[1,0,0,1,0]] → 4
```

⚡ dp[i][j]=1+min(左上,上,左)，取全部 dp 最大值平方。

右下角 dp = 1 + min(左, 上, 左上)，即能扩的最大边长



粉色=1 绿色=0

2×2 正方形
dp=2

min(左上=1,上=1,左=1)+1
= 2 → 2×2 边长

取 max dp=2 → 面积 4
O(m×n) 一维滚动即可

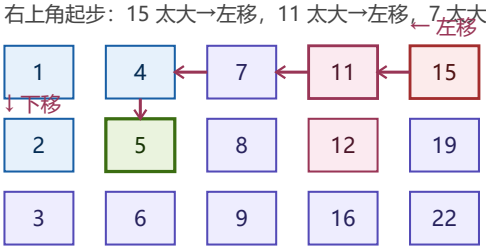
#240 搜索二维矩阵 II 中等 · 二分 / 分治 / 矩阵

每行、每列都递增的矩阵中查找 target。

```
matrix = [[1,4,7,11,15],
          [2,5,8,12,19],
          [3,6,9,16,22],
          [10,13,14,17,24],
          [18,21,23,26,30]]
target = 5 → true
```

⚡ 从右上角出发：比 target 大就左移，比 target 小就下移，O(m+n) 一条路走完。

右上角起步：15 太大→左移，11 太大→左移，7 太大→左移，4 太小→下移，5 命中



规则：cur > target → 左移
cur < target → 下移

找到 target=5 → true

为什么右上角：它是行最大+列最小的「分水岭」
每一步排除一整行或一整列，保证不回头

链表

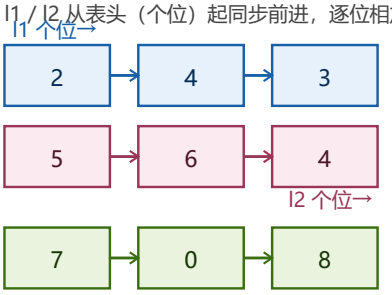
#2 两数相加 中等 · 链表 / 递归 / 数学

两个逆序存储数字的链表，逐位相加并进位，返回结果链表。

```
l1 = [2,4,3]
l2 = [5,6,4]
→ [7,0,8]
（342+465=807）
```

⚡ 同步遍历两链，逐位和 + 进位 carry；末位若 carry=1 补一个新节点。

l1 / l2 从表头（个位）起同步前进，逐位相加



2+5=7 进位 0

4+6=10 → 写 0 进 1

3+4+进位1=8，无进位，结束

carry 全程只可能 0/1，最终若仍为 1 需补节点 1

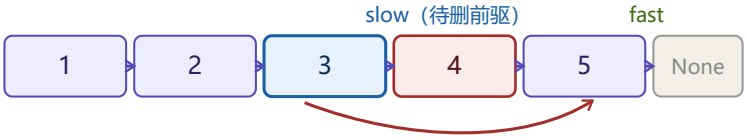
#19 删除链表的倒数第 N 个结点 中等 · 链表、双指针

删除链表的倒数第 n 个结点，返回头结点。

```
输入：head = [1,2,3,4,5], n = 2
输出：[1,2,3,5]
```

⚡ 快慢双指针：fast 先走 n 步，再与 slow 同步走，fast 到末尾时 slow 停在待删结点的前驱。

fast 先走 n=2 步，然后两指针同步移动



slow.next = slow.next.next → 跳过节点 4，完成删除

返回 head；新增哑结点（dummy）可统一删头与删中情况

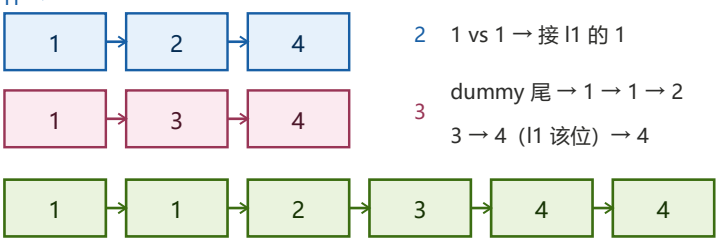
#21 合并两个有序链表 简单 · 链表 / 递归

把两条升序链表合并成一条新的升序链表。

```
l1 = [1,2,4]
l2 = [1,3,4]
→ [1,1,2,3,4,4]
```

⚡ 哑节点 dummy 接较小者，谁小接谁，指针后移；一方耗尽直接接剩余整段。

dummy 尾指针逐次接 l1/l2 中更小者



2 1 vs 1 → 接 l1 的 1

3 dummy 尾 → 1 → 1 → 2
3 → 4（l1 该位）→ 4

稳定 & 高效 O(m+n)：每次只比较两链当前头，恒接较小者

#23 合并 K 个升序链表 困难 · 链表、分治、堆

K 个有序链表合并成一个有序链表。

```
输入：lists = [[1,4,5],[1,3,4],[2,6]]
输出：[1,1,2,3,4,4,5,6]
```

⚡ 最小堆：K 个头入堆，每次弹最小节点接上，再将其 next 入堆。

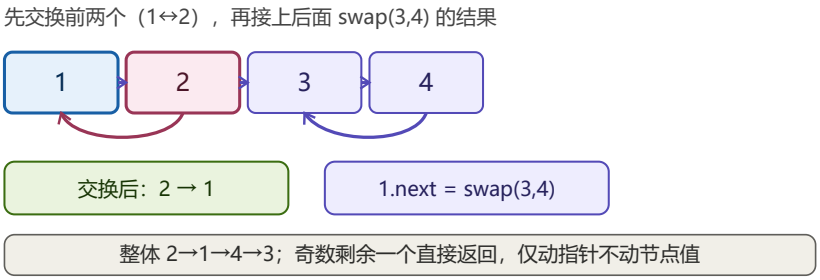


#24 两两交换链表中的节点 中等 · 递归、链表

每两个相邻节点交换位置，不修改节点内部值，返回新头。

```
输入：head = [1,2,3,4]
输出：[2,1,4,3]
```

⚡ 递归：swap(head) = 头变第2个，第1个接上 swap(第3个起)；只换指针不换值。

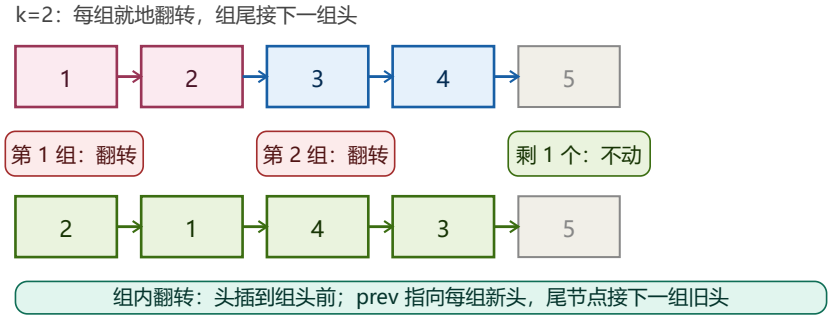


#25 K 个一组翻转链表 困难 · 链表 / 递归

每 k 个节点一组翻转，不足 k 个保持原序。

```
head = [1,2,3,4,5], k = 2
→ [2,1,4,3,5]
head = [1,2,3,4,5], k = 3
→ [3,2,1,4,5]
```

⚡ 先数够 k 个才翻；组内头插法翻转，prev 接住各组，group 尾连下一组头。

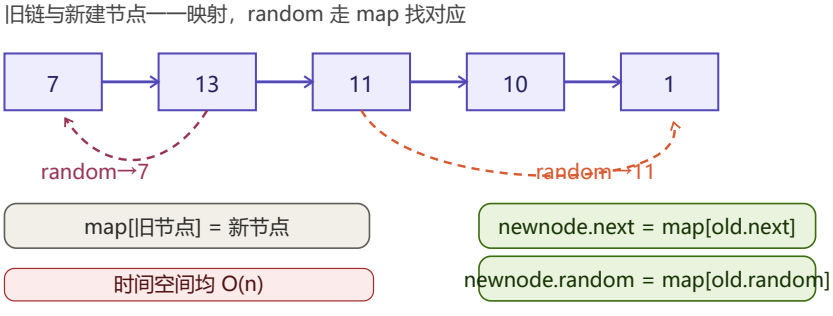


#138 随机链表的复制 中等 · 链表 / 哈希表

深拷贝每条 next 与 random 指针对应的链表。

```
head = [[7,null],[13,0],[11,4],[10,2],
[1,0]]
→ 同结构新链（random 全指向新节点）
```

⚡ 哈希表 old→new：第一遍建节点，第二遍补 next 与 random 指向。



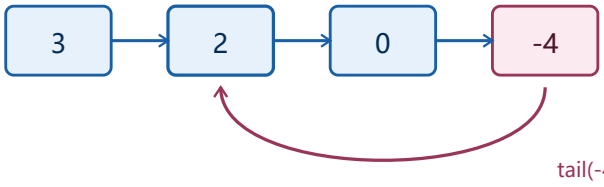
#141 环形链表 简单 · 哈希表、链表、双指针

判断链表中是否有环。

```
输入：head = [3,2,0,-4], pos = 1
输出：true
解释：尾部连接到下标 1 的节点，成环
```

⚡ 快慢指针：slow 走 1 步 fast 走 2 步，有环必相遇；无环 fast 先到 null。

3→2→0→−4，tail 接回 2 号（下标 1）形成环



slow (1x) 与 fast (2x) 同时出发
有环 → fast 在环内追上 slow（绕圈差 1 步/轮）

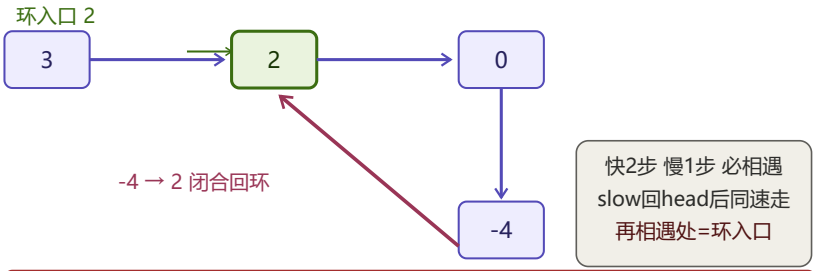
#142 环形链表 II 中等 · 链表、双指针、Floyd 判圈

返回入环的第一个节点；无环返回 null。

```
输入：head = [3,2,0,-4], pos = 1
输出：返回索引 1 的节点
```

⚡ Floyd：快慢指针相遇后，慢指针从头再出发，快慢同速走，再次相遇点即环入口。

3 → 2 → 0 → -4 → 2（回到入口 2，成环）



快慢同速再走，二者再相遇的位置就是环入口；无环则返回 null

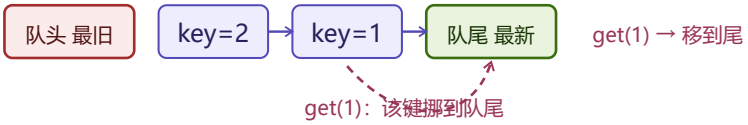
#146 LRU 缓存 中等 · 设计 / 哈希表 / 双向链表

get/put 均 O(1)：容量满时淘汰最久未使用的键。

```
capacity=2
put(1,1) put(2,2)
get(1) → 1
put(3,3) 挤出 key 2
get(2) → -1
```

⚡ 哈希表定位 + 双向链表维护顺序：新用放队尾，满时删队头（最久未用）。

LRU：队尾最新，队头最旧，满则删队头



put(3,3)：满 → 删队头 key=2，再在队尾插 key=3

哈希 map 提供 O(1) 定位，双向链表提供 O(1) 增删

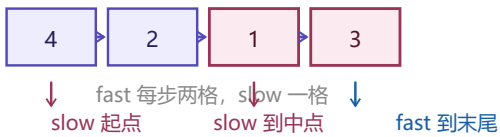
#148 排序链表 中等 · 链表 / 双指针 / 分治 / 归并排序

对链表 O(nlogn) 排序（不能用随机访问）。

```
head=[4,2,1,3] → [1,2,3,4]
```

⚡ 归并：快慢指针找中点拆两半，递归排序后合并两条有序链。

快慢指针找中点，切分成两半 → 递归归并



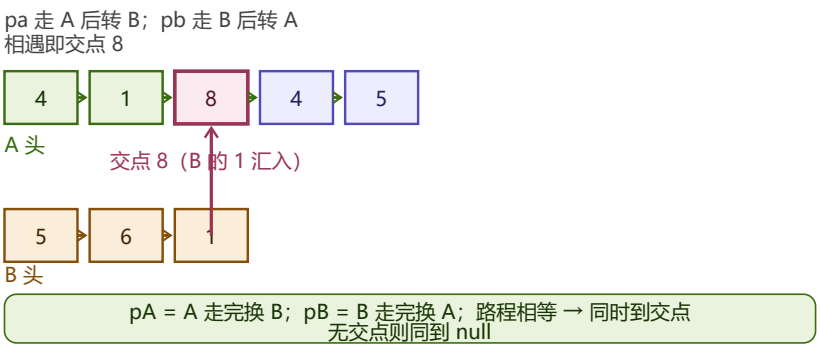
两半：左 [4,2] 右 [1,3]，各递归归并排序
合并两个有序链（双指针比小），O(nlogn)、O(1) 空间

#160 相交链表 简单 · 哈希表 / 链表 / 双指针

找两个链表相交的起始节点。

```
A=[4,1,8,4,5], B=[5,6,1,8,4,5] → 节点 8
```

⚡ 双指针各走自己的链再换链，走过的路程相等时必在交点相遇。

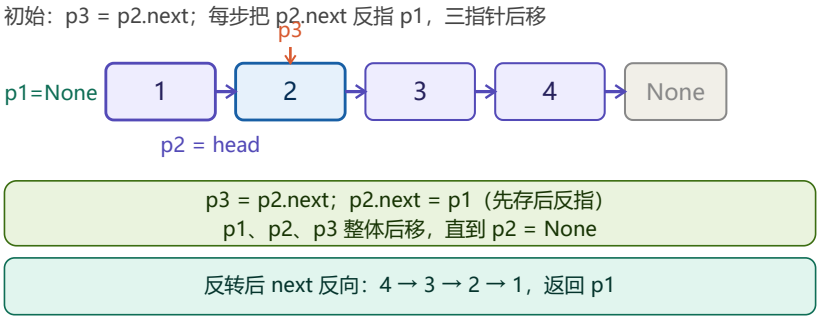


#206 反转链表 简单 · 递归、链表

反转单链表，返回反转后的头节点。

```
输入：head = [1,2,3,4,5]
输出：[5,4,3,2,1]
```

⚡ 三指针迭代：p1 是已反转链的头，p2 是待处理节点，每步把 p2.next 指回 p1 再整体后移。

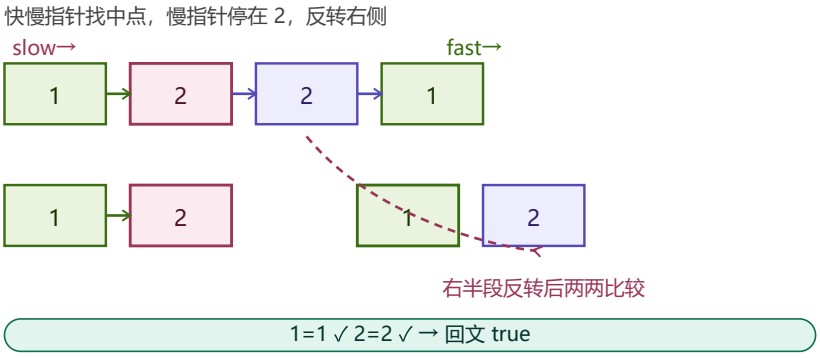


#234 回文链表 简单 · 链表 / 双指针

判断链表是否为回文（正读反读相同）。

```
head = [1,2,2,1]
→ true
head = [1,2]
→ false
```

⚡ 快慢指针找中点 → 反转后半段 → 两半逐位比较；空间 O(1)。



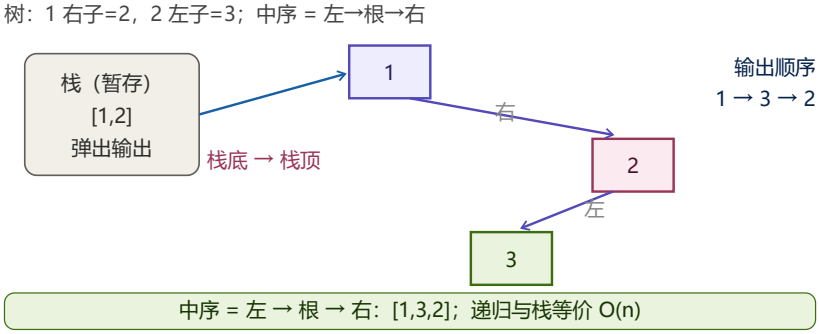
二叉树

#94 二叉树的中序遍历 简单 · 栈 / 树 / DFS / 二叉树

中序遍历：左 → 根 → 右。

```
[1,null,2,3] → [1,3,2]
```

⚡ 一直往左扎进栈，弹出即输出，再转右子树；每节点进栈一次。



#98 验证二叉搜索树 中等 · 树、DFS、二叉搜索树

判断二叉树是否为有效 BST：左子树全小于根、右子树全大于根。

```
输入：root = [2,1,3]
输出：true
```

⚡ 区间约束递归：子树递归时带上 (min, max) 上下界，节点值越界即非法；空节点为真。

向下传递区间：左子 (min, root.val)，右子 (root.val, max)



检查：1 ∈ (-∞,2) ✓；3 ∈ (2,+∞) ✓ → 合法 BST

若某个节点值不落在其区间内 → 立即返回 false（含左右子树各层约束）

中序遍历等价值：BST 中序一定是严格递增序列

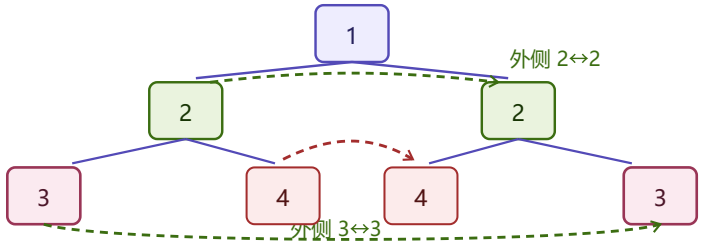
#101 对称二叉树 简单 · 二叉树 / 递归

检查二叉树是否关于中轴镜像对称。

```
root = [1,2,2,3,4,4,3]
→ true
root = [1,2,2,null,3,null,3]
→ false
```

⚡ 比较 left.left 与 right.right、left.right 与 right.left，递归成对判断。

镜像比较：外侧比外侧（2↔2、3↔3），内侧比内侧（4↔4）



两个 2 对称 ✓、3=3 ✓、4=4 ✓ → 整棵树对称

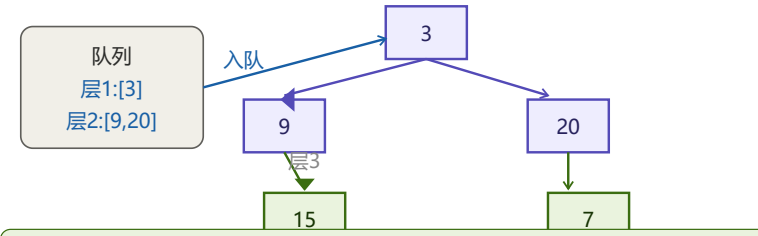
#102 二叉树的层序遍历 中等 · 树 / BFS / 二叉树

按层从左到右逐层输出节点值。

```
[3,9,20,null,null,15,7] → [[3],[9,20],
[15,7]]
```

⚡ BFS：用队列逐层入队，每轮先取「当前队列长度」= 本层节点数。

队列：先取整层，出队时把子节点入队



关键：for 本层 len 次出队；子节点入队留到下一层处理

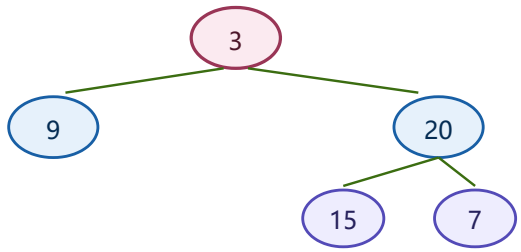
#104 二叉树的最大深度 简单 · 树、DFS、BFS、二叉树

给定二叉树根节点，返回最大深度。

```
输入：root = [3,9,20,null,null,15,7]
输出：3
```

⚡ 递归：depth = 1 + max(左深, 右深)；空节点返回 0。

每层 +1，取左右子树较深者



路径 3→20→15/7 最深：1+max(0, 1+1)=3

#105 从前序与中序遍历构造二叉树 中等 · 树、哈希表、分治

用前序+中序遍历重建二叉树。

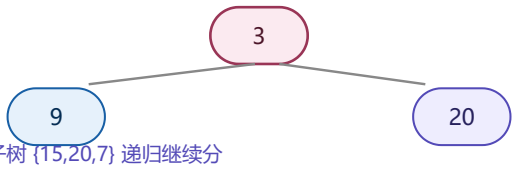
```
输入：preorder = [3,9,20,15,7], inorder = [9,3,15,20,7]
输出：[3,9,20,null,null,15,7]
```

⚡ 前序首位定根，中序按根切左右子树，哈希表记住中序下标 O(1) 定位，递归分治。

前序[0]=3 是根 → 中序里 3 左侧 {9} 为左子树，右侧 {15,20,7} 为右子树



前序 [3, 9, 20, 15, 7]　中序 [9, 3, 15, 20, 7] 根=3



左子树 {9}；右子树 {15,20,7} 递归继续分

#108 将有序数组转换为二叉搜索树 简单 · 二叉树 / 分治

把升序数组构造成高度平衡的二叉搜索树。

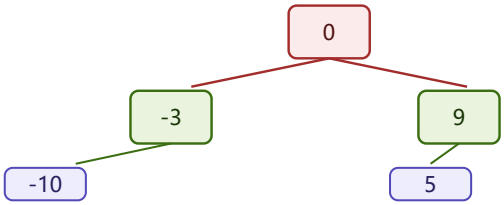
```
nums = [-10,-3,0,5,9]
→ [0,-3,9,-10,null,5]
   0
  / \
-3   9
/   /
-10 5
```

⚡ 取中点作根，左半边递归建左子树、右半边建右子树，天然平衡。

每次取中点当根 → 左右区间递归



mid → 根



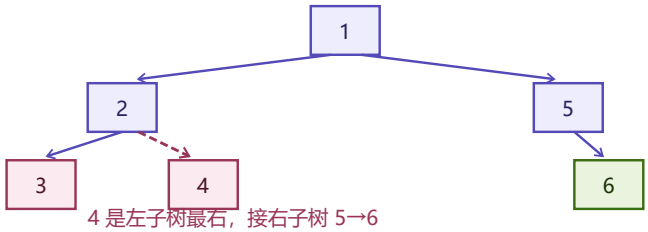
#114 二叉树展开为链表 中等 · 栈 / 树 / DFS / 链表 / 二叉树

把二叉树按「前序」顺序展开成右链形式的单链表。

```
[1,2,5,3,4,null,6] →
[1,null,2,null,3,null,4,null,5,null,6]
（前序 1→2→3→4→5→6 串成右链）
```

⚡ Morris 式：找左子树最右节点，把右子树接过去；左子树搬到右侧，左置空。

原树：把 2 的右子树接到 4 之下 → 链 1→2→3→4→5→6



4 是左子树最右，接右子树 5→6

然后 2 移到 1 的右侧，左指针置 null；前序自然成立

#124 二叉树中的最大路径和 困难 · 树形 DP / DFS

路径可起止于任意节点、向上只能走一条分支，求最大路径和。

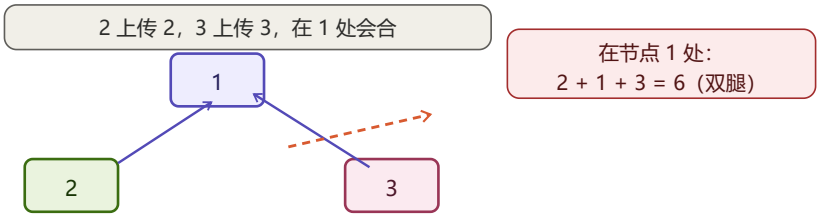
```
root = [1,2,3]
→ 6（路径 2→1→3）
root = [-10,9,20,
        null,null,15,7]
→ 42（路径 15→20→7）
```

⚡ 后序 DFS：每节点算「单腿贡献」max(child,0)+val 向上传，答案 = 节点 + 左单腿 + 右单腿的最大值。

单腿向上传，双腿在节点交汇成完整路径

贡献 = 节点 + max(子贡献, 0)

2 上传 2，3 上传 3，在 1 处会合



在节点 1 处：
2 + 1 + 3 = 6（双腿）

负子树贡献取 0（剪掉）；答案 ans = max(ans, L+R+val) 全程更新

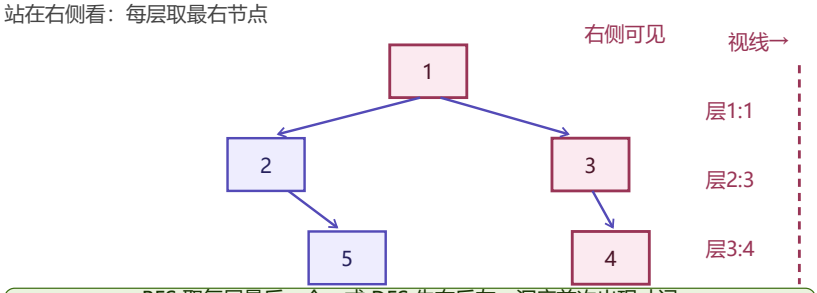
#199 二叉树的右视图 中等 · 树 / DFS / BFS / 二叉树

从右侧看二叉树，返回每层最右边的节点值。

```
[1,2,3,null,5,null,4] → [1,3,4]
```

⚡ BFS 每层取最右节点；或 DFS 右→左，首次到达每层深度时记录。

站在右侧看：每层取最右节点



BFS 取每层最后一个；或 DFS 先右后左，深度首次出现才记

#226 翻转二叉树 简单 · 树、DFS、BFS、二叉树

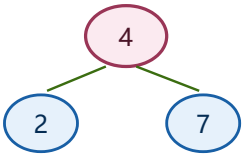
翻转二叉树，即交换每个节点的左右子树。

```
输入：root = [4,2,7,1,3,6,9]
输出：[4,7,2,9,6,3,1]
```

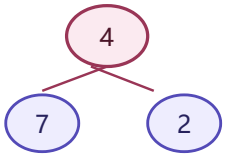
⚡ 递归：先翻转左右子树，再交换左右孩子；空节点直接返回。

递归到叶子：交换每个节点的左右子树

翻转前



翻转后（左右镜像）



交换每个节点左右子树：4 → 左7右2，下一层 9 6 3 1
伪码：l=invert(left); r=invert(right); 再交换 root.left / root.right

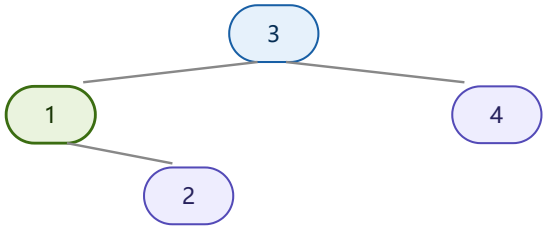
#230 二叉搜索树中第 K 小的元素 中等 · 树、DFS、二叉搜索树

在 BST 中找第 k 小的元素（k 从 1 计数）。

```
输入：root = [3,1,4,null,2], k = 1
输出：1
```

⚡ 中序遍历 BST 得到升序序列，第 k 个访问的节点就是答案；可配合左子树节点数剪枝。

中序遍历 = 从小到大：左 → 根 → 右



中序序列：[1, 2, 3, 4] → 第 k=1 个 = 1
计数器 cnt 到 k 即停；k > 左子树大小 → 直接进右子树（剪枝优化）

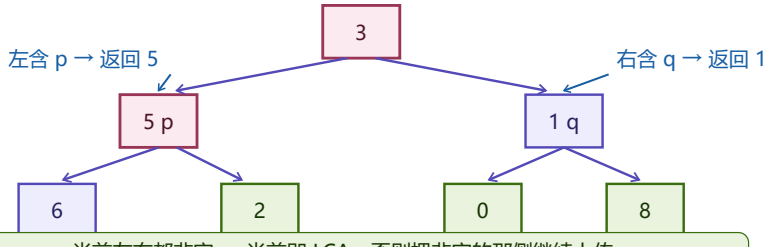
#236 二叉树的最近公共祖先 中等 · 树 / DFS / 二叉树

找两个指定节点 p、q 的最近公共祖先。

```
root=[3,5,1,6,2,0,8,null,null,7,4], p=5,
q=1 → 3
```

⚡ 递归后序：左子树有 p/q、右子树有 q/p 或自身就是目标时，返回当前节点。

自底向上找：左右都找到就返回当前节点



当前左右都非空 → 当前即 LCA；否则把非空的那侧继续上传

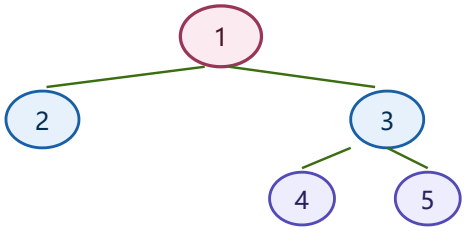
#297 二叉树的序列化与反序列化 困难 · 树、DFS、BFS、设计、字符串

把二叉树转成字符串再还原，任意方法可行。

```
输入：root = [1,2,3,null,null,4,5]
输出：[1,2,3,null,null,4,5]
```

⚡ 前序 + 空节点标记：# 代表 null；反序列化按同一前序顺序重建。

前序遍历序列化：1,2,#,#,3,4,#,#,5,#,#（#=空）



反序列化：按前序队列顺序建节点，读到 # 返回空

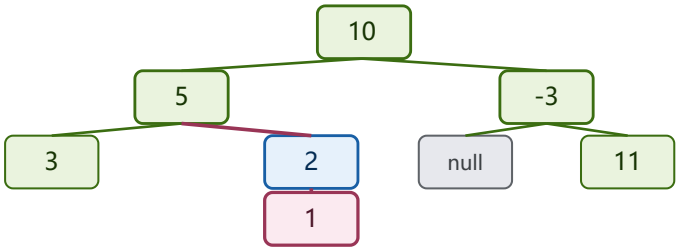
#437 路径总和 III 中等 · 树、DFS、前缀和

给定二叉树与目标值 targetSum，求节点值之和等于 targetSum 的向下路径数目（任意起点，沿父子方向延伸）。

```
输入：root =
[10,5,-3,3,2,null,11,3,-2,null,1],
targetSum = 8
输出：3 （5+3 / -3+11 / 5+2+1）
```

⚡ 前缀和 + 哈希：DFS 时查「当前前缀和 − targetSum」出现过几次；回溯时撤销，防止串到兄弟子树。

前缀和 cur = 根到当前节点之和，查 cur − targetSum 出现次数



红色 = 5+2+1=8 命中路径；回溯时删当前前缀和

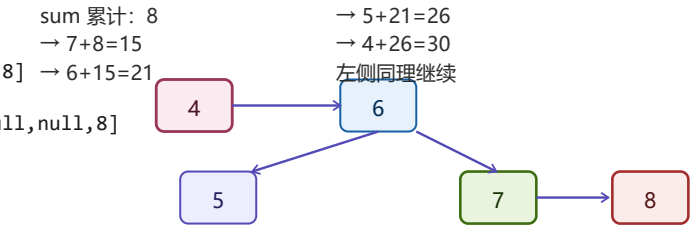
#538 把二叉搜索树转换为累加树 中等 · BST / 中序变体

BST 每个节点改为「原值 + 所有比它大的节点值之和」。

```
root =
[4,1,6,0,2,5,7,null,null,null,3,null,null,null,8]
→
[30,36,21,36,35,26,15,null,null,null,33,null,null,null,8]
```

⚡ 反中序遍历（右→中→左）用全局 sum 累加：先右子树加 sum，再节点 += sum，再左子树。

反中序顺序 = 从大到小：8→7→6→5→4（sum 一路累加）



中序天然升序，反中序天然降序 → 一次遍历完成累加

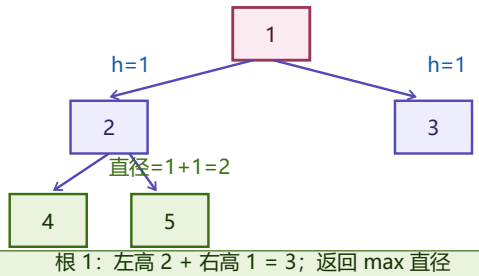
#543 二叉树的直径 简单 · 树 / DFS / 树形 DP

任意两节点间最长路径的边数。

```
[1,2,3,4,5] → 3（路径 4→2→1→3）
```

⚡ 后序返回每个节点的高度，直径候选 = 左高 + 右高，全程取 max。

每个节点：直径候选 = 左子树高度 + 右子树高度



根 1：左高 2 + 右高 1 = 3；返回 max 直径

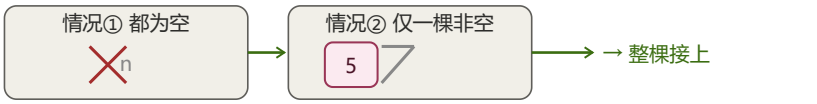
#617 合并二叉树 简单 · 二叉树 / 递归

两棵树对应位置相加，重合节点求和，空缺处保留另一棵。

```
root1 = [1,3,2,5]
root2 = [2,1,3,null,4,null,7]
→ [3,4,5,5,4,null,7]
```

⚡ 递归三情况：都空返回空；单空返回另一棵；双在则值相加，再递归合并左右子。

merge(t1, t2) 递归三情况



情况③ 两棵都在：新节点值 = 两值相加

3 + 7 = 10　左右子树继续递归 merge

示例：根 1+2=3，左侧 3+1=4，再下层 5+null=5、null+4=4

图论

LeetCode 可视复习卡 · 图论/DFS

题面 + 示例 + 核心套路可视化 + 一句话口诀
(2026-09-26)

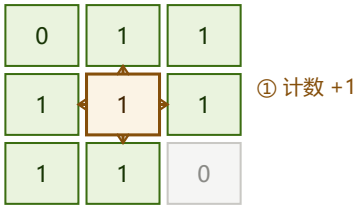
#200 岛屿数量 中等 · 深度优先搜索 / 广度优先搜索 / 并查集 / 矩阵

'1' 表示陆地、'0' 表示水，统计被水隔开的连通陆地块数。

```
grid = [
  ['1','1','1','1','0'],
  ['1','1','0','1','0'],
  ['1','1','0','0','0'],
  ['0','0','0','0','0']
]
→ 输出：1
```

⚡ 主循环遇 '1' 计数并 DFS 淹没四邻（改'0'），每格只访问一次，统计块数。

遇 '1' 计数 +1，DFS 沿上/下/左/右四方向淹没邻接陆地



① 计数 +1

DFS 淹没：遇 '1' 继续递归四邻，
遇 '0' 或出界返回，并置当前格为 '0'

→ 同一块陆地只被主循环计数 1 次

主循环遍历每个格子：遇 '1' 计数并 DFS 改 '0'；最后总计数 = 岛屿数

#207 课程表 中等 · DFS、BFS、图、拓扑排序

numCourses 门课有先修关系，判断能否学完全部课程。

```
输入：numCourses = 2, prerequisites =
[[1,0]]
输出：true
解释：学 1 前需完成 0，无环可行
```

⚡ 拓扑排序判环：统计入度，入度 0 的课先上；若有环则学不完。

边 0→1：先修 0 再修 1（有向无环可行）



入度 0 → 入队
学 0 → 1 入度减为 0 → 学 1

学的课程数 = 节点数 → 无环可完成；否则说明有环

有环例子：[[0,1],[1,0]] 互先修 → 学不完 → false

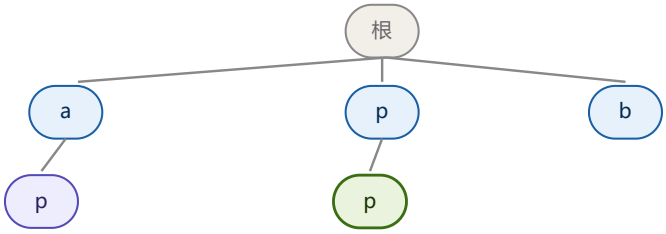
#208 实现 Trie（前缀树） 中等 · 设计、字典树、哈希表、字符串

实现 insert / search / startsWith 三种操作的前缀树。

```
输入:
["Trie","insert","search","startsWith"],
[[],["apple"],["apple"],["app"]]
输出: [null,null,true,true]
```

⚡ 每个节点 26 个子指针 + isEnd 标记；按字符逐层走，末尾打上结束标记。

从根开始逐字符走；每字符一层的边，末尾节点打 isEnd 标记



"app" 结束（isEnd）→ search 命中；"appl..." 仅前缀 → startsWith true

LeetCode 可视复习卡 · 带权图

题面 + 示例 + 核心套路可视化 + 一句话口诀
(2026-09-26)

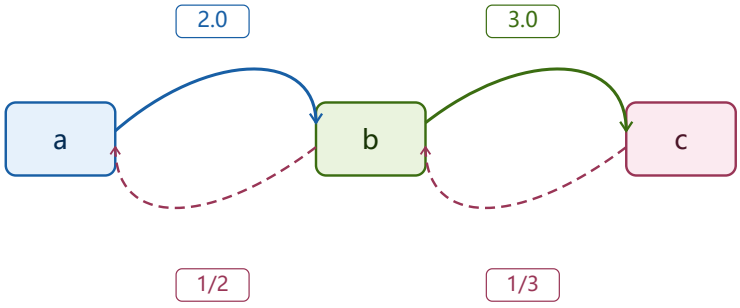
#399 除法求值 中等 · 深度优先搜索 / 图 / 并查集

等式 Ai/Bi = values[i]，回答若干除法查询，无法推出返回 -1.0。

```
equations = [["a","b"],["b","c"]]
values = [2.0, 3.0]
queries = [["a","c"],["b","a"],
           ["a","e"],["a","a"],["x","x"]]
→ 输出: [6.0, 0.5, -1.0, 1.0, -1.0]
```

⚡ 每条除法看作带权边：A→B 权 w、B→A 权 1/w；DFS 沿路径连乘边权。

带权图：a→b 权 2.0，b→c 权 3.0；反向边取倒数



a/c = 6.0，b/a = 0.5，a/a = 1.0，无路径 → -1.0

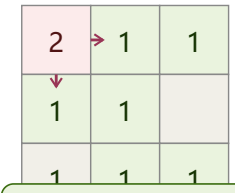
#994 腐烂的橘子 中等 · 广度优先搜索 / 数组 / 矩阵

烂橘子每分钟向四邻传染，求全部烂掉所需分钟。

```
[[2,1,1],[1,1,0],[0,1,1]] → 4
```

⚡ 多源 BFS：先入队所有烂橘子，层数即分钟数；检查是否还有新鲜橘子。

多源 BFS：每分钟向外一圈传播，层数=分钟数



2 向四邻传染（→ 右、↓ 下）
传染到的新烂橘子在下一圈再扩散
第1圈→第2圈→...→全烂所需圈数 = 分钟数

BFS 结束时若还有 1 → 返回 -1（永远烂不到）
ans = 最后一层的层号

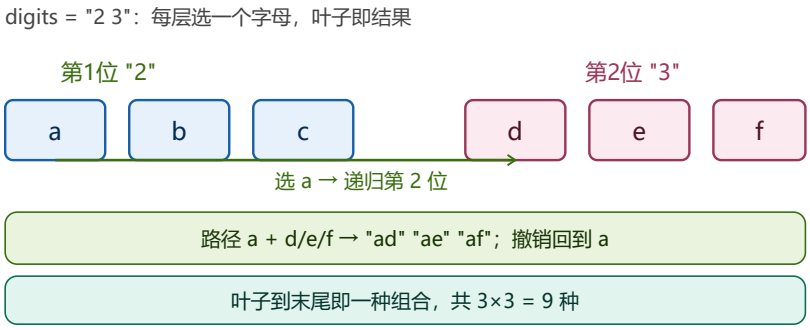
回溯

#17 电话号码的字母组合 中等 · 哈希表、字符串、回溯

给定数字串，返回每个数字对应字母的所有组合。

```
输入：digits = "23"
输出：
["ad","ae","af","bd","be","bf","cd","ce","cf"]
```

⚡ 回溯：逐位选一个字母，递归到最后一位收集结果，再撤销回溯。

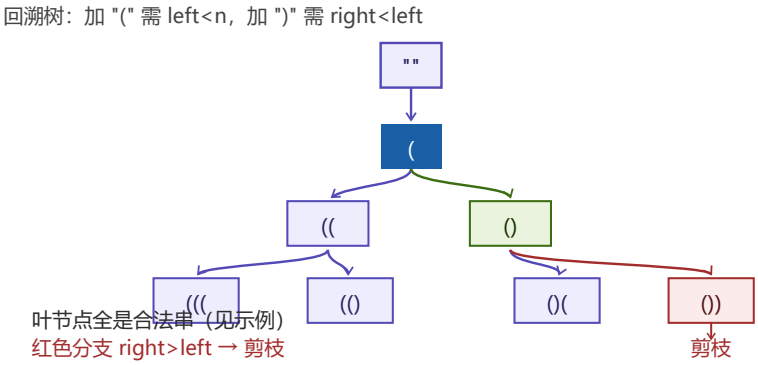


#22 括号生成 中等 · 字符串 / 动态规划 / 回溯

生成 n 对括号的所有「有效」组合。

```
n=3 → ["((()))","(()())","(())()","()
(())","()()()"]
```

⚡ 回溯：左≤n、右≤左才合法，剪枝无效分支。

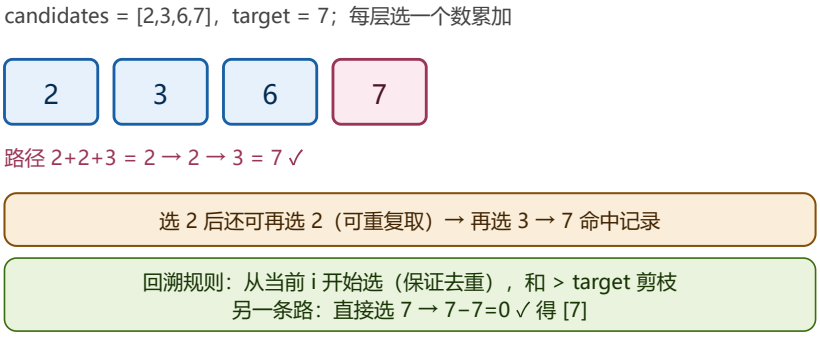


#39 组合总和 中等 · 数组、回溯

从候选数中选（可重复）使和等于 target，返回所有组合。

```
输入：candidates = [2,3,6,7], target = 7
输出：[[2,2,3],[7]]
```

⚡ 回溯：从当前位置起选一个数加入路径，剩余 target 减掉；可重复选同一位置。

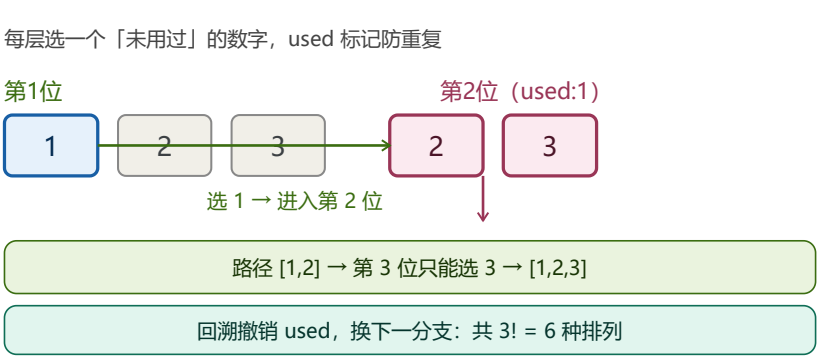


#46 全排列 中等 · 数组、回溯

返回不含重复数字数组的所有排列。

```
输入：nums = [1,2,3]
输出：[[1,2,3],[1,3,2],[2,1,3],[2,3,1],
[3,1,2],[3,2,1]]
```

⚡ 回溯 + used 数组：每层从「未用」数字挑一个，递归到底收集，再撤销标记。



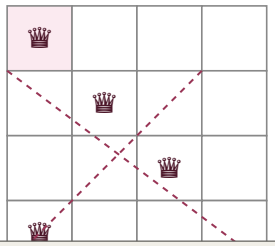
#51 N 皇后 困难 · 数组 / 回溯

n×n 棋盘放 n 个皇后，同行同列同斜线互不攻击，返回所有摆法。

```
n=4 → [[".Q..","...Q","Q...","..Q."],
["..Q.","Q...","...Q",".Q.."]]
```

⚡ 逐行回溯：用 列 / 主对角 / 副对角 三个集合判冲突，放不下就回退。

4×4 棋盘，逐行放皇后，三种冲突线都要避让



冲突：同列 / 主对角线 / 副对角线

列 c

主对角 r-c

副对角 r+c

回溯：放不下→撤销，换下一列

深度优先，行满即一条解

三集合 O(1) 判冲突，剪掉整棵子树

#78 子集 中等 · 位运算、数组、回溯

返回数组所有子集（幂集），元素互不相同。

```
输入：nums = [1,2,3]
输出：[[],[1],[2],[1,2],[3],[1,3],[2,3],
[1,2,3]]
```

⚡ 每个元素两种选择：选 / 不选；DFS 到末尾时收集路径；或位掩码枚举。

回溯树：每个元素分「选 / 不选」两支，叶子就是子集

1 2 3

对 1：选 → 1 进路径；不选 → 跳过

对 2、3 同理，逐元素决定后收集叶

路径示例：[1]

每个数两种选择 → 叶子共 $2^3 = 8$ 个子集（含空集）

位掩码等价：3 位二进制 000 ~ 111 一一对应 8 个子集

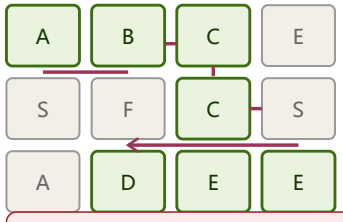
#79 单词搜索 中等 · DFS、回溯、矩阵

在 m×n 网格中找单词（相邻四方向连接，字母不可重复用）。

```
输入：board = [['A','B','C','E'],
['S','F','C','S'],['A','D','E','E']],
word = "ABCCED"
输出：true
```

⚡ 回溯：从匹配首字母的格子出发 DFS 上下左右，走过的格临时标记访问，失败回退还原。

word = A-B-C-C-E-D，按字母顺序相邻搜索



A→B→C→C(下)→E→D 依次命中

访问过的格子做临时标记，

当前分支失败时撤销标记（回溯）

四个方向都可以试

失败即回退（如 C 向下走不通就回 C 试下一条路），全字母匹配完返回 true

注意：一个格子不能复用，起点需遍历所有与首字母相同的格子

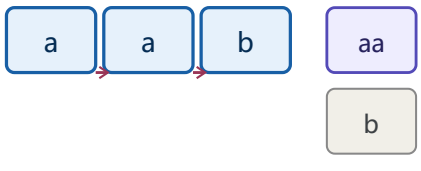
#131 分割回文串 中等 · 字符串、动态规划、回溯

把字符串 s 分割成若干子串，每个都是回文，返回所有方案。

```
输入：s = "aab"
输出：[["a","a","b"],["aa","b"]]
```

⚡ 回溯：从当前下标切出所有回文前缀，递归剩余部分，回溯撤销。

s = "aab"：在 a|a|b 与 aa|b 两处切分



"aa" 回文 ✓

方案1：a|a|b

方案2：aa|b

回溯：start 处枚举回文前缀，递归 start+len

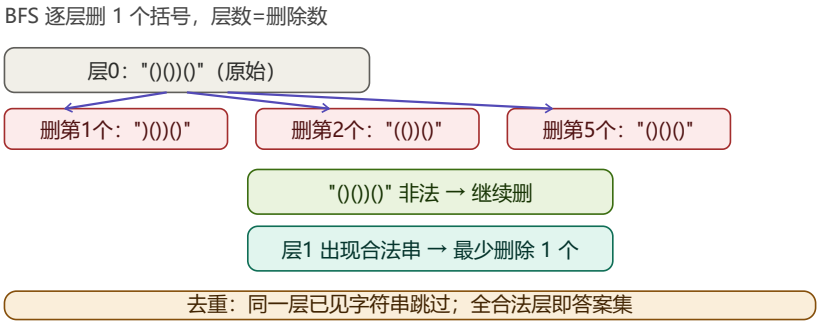
回文判断可用中心扩展或预计算 dp[i][j]

#301 删除无效的括号 困难 · BFS / 回溯

删除最少数量的括号使字符串有效，返回所有结果。

```
s = "()())()"
→ ["(())()", "()()()"]
s = "(a())()"
→ ["(a())()", "(a)()()"]
```

⚡ BFS 按层删除：每层删 1 个括号，逐层扩张；首个出现合法字符串的层即最少删除层。



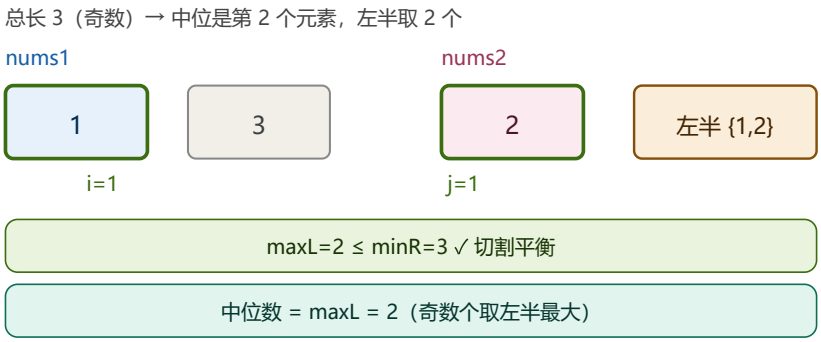
二分查找

#4 两有序数组中位数 困难 · 二分查找、分治

两个有序数组，找合并后的中位数，要求 O(log(m+n))。

```
nums1 = [1,3], nums2 = [2]
合并 [1,2,3] → 中位数 2.0
```

⚡ 二分切短数组：切 i、另一处 j = 总数/2 − i，使左半 ≤ 右半即得中位数。



#33 搜索旋转排序数组 中等 · 数组、二分查找

在旋转后的升序数组中找 target，数组值互不相同。

```
输入：nums = [4,5,6,7,0,1,2], target = 0
输出：4
```

⚡ 变体二分：先判 mid 在左/右哪段有序，再按 target 与边界比较决定收缩方向。

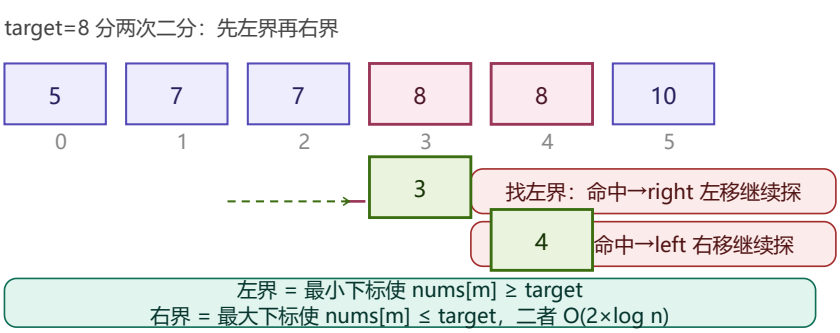


#34 在排序数组中查找元素的第一个和最后一个位置 中等 · 二分查找 / 数组

在非降序数组中查 target 的起止下标；不存在返回 [-1,-1]。

```
nums = [5,7,7,8,8,10], target = 8
→ [3,4]
```

⚡ 两次二分：第一次找左边界（相等时收缩 right），第二次找右边界（相等时收缩 left）。



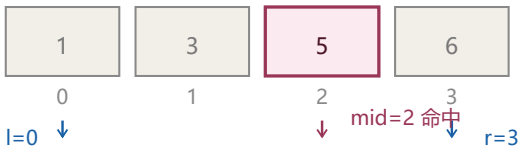
#35 搜索插入位置 简单 · 数组 / 二分查找

在升序数组中找 target 下标，不存在则返回应插入的位置。

```
nums=[1,3,5,6], target=5 → 2
```

⚡ 标准二分：left 收敛到第一个 ≥ target 的位置，正好是答案。

nums=[1,3,5,6] target=5，二分区间 [l, r]



命中→返回 mid；未命中→l 指向第一个 ≥ target 处，即插入位

O(log n)，左闭右开二分模板：l<r，l 收敛为答案

#74 搜索二维矩阵 中等 · 数组、二分查找、矩阵

每行升序、每行首元素大于上行末元素，判断 target 是否存在。

```
输入：matrix = [[1,3,5,7],[10,11,16,20],
[23,30,34,60]], target = 3
输出：true
```

⚡ 逻辑一维化：把矩阵拉成长度为 m×n 的升序数组，一次标准二分；编号→行列：row = mid/n, col = mid%n。

整矩阵按行展开 = 一个升序数组，直接二分



全部 12 个数拉平为一维严格递增序列（行尾 < 下步行首）

二分 target=3：11>3 砍右 → 5>3 砍右 → 1<3 → 3 命中

编号转行列：mid=5 → row=⌊5/4⌋=1, col=5%4=1 → 值 11
一维下标即可二分；mid 映射 (row,col) 访问矩阵元素

整体 O(log(m·n))，等价标准二分在虚拟长数组上运行

#153 寻找旋转排序数组中的最小值 中等 · 数组、二分查找

在旋转后的升序数组中找最小元素（元素互不相同）。

```
输入：nums = [3,4,5,1,2]
输出：1
解释：原数组 [1,2,3,4,5] 旋转 3 次
```

⚡ 比较 nums[mid] 与 nums[r]：mid 更大搜右半，反之搜左半；区间一直收缩。

每次对比 nums[mid] 与 nums[r]，决定砍左半还是右半



mid=5 > nums[r]=2 → 最小值在右半，l=mid+1 → 区间 [1,2]
再查 mid(下标3)=1：1 < 2 → r=mid；最终锁在下标 3 → 返回 1

判据：nums[mid] > nums[r] → 搜右；否则 → 搜左（含 mid）

栈

LeetCode 可视复习卡 · 栈

题面 + 示例 + 核心套路可视化 + 一句话口诀
(2026-09-26)

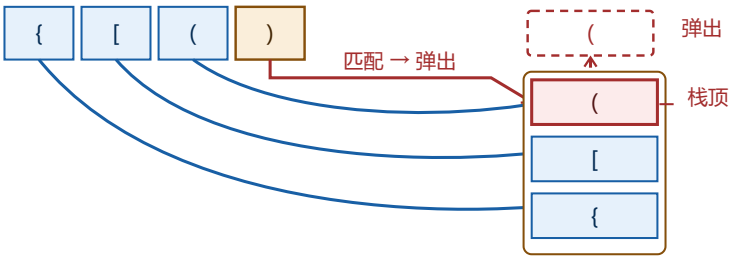
#20 有效的括号 简单 · 栈 / 字符串 / 括号序列

只含 ( ) { } [ ] 的字符串，判断括号是否成对、按正确顺序闭合。

```
s = "()" → true
s = "()[]{}" → true
s = "(]" → false
s = "([)]" → false
```

⚡ 左括号入栈；右括号必须与栈顶配对后弹出；遍历完且栈空才有效。

s = { [( ) — 右括号 ")" 到来，匹配栈顶后弹出



蓝弧：{ [(依次入栈，后入的 (在栈顶；红箭：) 匹配栈顶 → 弹出

遍历完①栈为空 → true；右括号不匹配栈顶或栈已空 → false

LeetCode 可视复习卡 · 单调栈

题面 + 示例 + 核心套路可视化 + 一句话口诀

(2026-09-26)

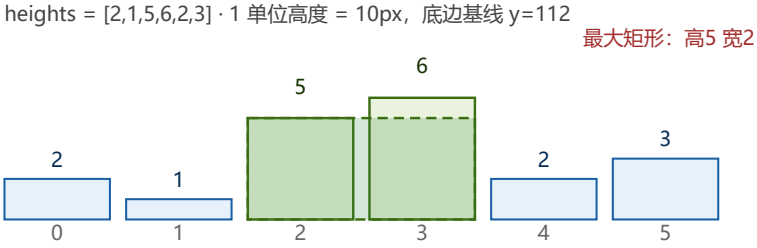
#84 柱状图中最大的矩形 困难 · 栈 / 数组 / 单调栈

n 根宽度为 1 的柱子，找能勾勒出的最大矩形面积。

```
heights = [2,1,5,6,2,3] → 10
最大矩形：高 5、宽 2（第 3、4 根柱子，
5×2=10）
```

⚡ 单调递增栈存下标：遇更矮柱子就出栈结算「高×左右边界距离」，结尾补 0 强制清栈。

heights = [2,1,5,6,2,3] · 1 单位高度 = 10px，底边基线 y=112



遇更矮柱子出栈结算；结尾补高度 0 强制清栈，每柱出入栈一次 O(n)

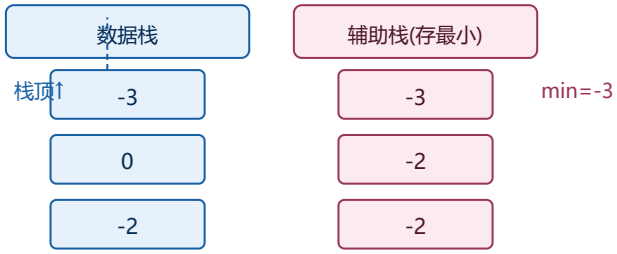
#155 最小栈 中等 · 栈 / 设计

栈支持 push/pop/top/getMin，getMin 要求 O(1)。

```
push(-2) push(0) push(-3)
getMin() → -3
pop() 弹出 -3 → top() → 0
getMin() → -2
```

⚡ 双栈或栈存 (值, 当前最小值)；每次 push 时同步记录入栈时刻的最小值。

数据栈 + 辅助栈（同进同出，辅助栈顶=当前最小）



getMin 直接读辅助栈顶；pop 时两栈同步 pop，栈与最小始终对齐

#394 字符串解码 中等 · 栈、递归、字符串

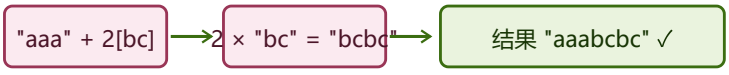
解码 k[encoded_string]，数字表示重复次数，可嵌套。

```
输入：s = "3[a]2[bc]"
输出："aaabcbc"
```

⚡ 栈：遇到] 弹出，把「数字 × 字符串」拼接回栈；数字和字符串分别用两个栈。

s = 3[a]2[bc]，遇] 弹栈 3 × "a" 得 "aaa"

步骤



栈帧：数字栈 [3, 2]，字符栈累积已解码段
遇到嵌套 [时，把外层数字和前缀先压栈

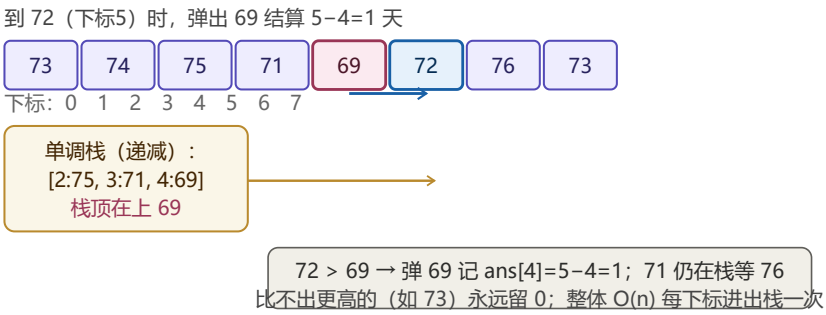
对 ']' 操作：pop 数字 × pop 字符串段 → 拼回段顶
数字可能多位：3[a2[c]] 先算 2[c] 再算 3×"acc"

#739 每日温度 中等 · 栈、单调栈

返回每个温度后第一个更高温度出现在几天后，否则 0。

```
输入：temperatures =
[73,74,75,71,69,72,76,73]
输出：[1,1,4,2,1,1,0,0]
```

⚡ 单调递减栈存下标：新温度更高时，弹出栈顶并结算其等待天数（栈顶与新下标之差）。



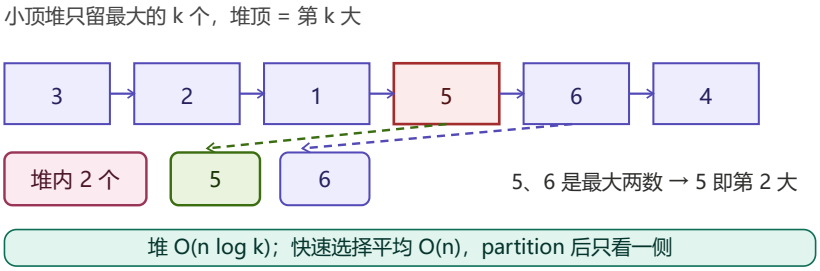
堆

#215 数组中的第 K 个最大元素 中等 · 快速选择 / 堆

无序数组找第 k 大的元素（不要求整体排序）。

```
nums = [3,2,1,5,6,4], k = 2
→ 5
nums = [3,2,3,1,2,4,5,5,6], k = 4
→ 4
```

⚡ 小顶堆保持 k 个元素，堆顶就是答案；或快速选择每轮只进一半区间。

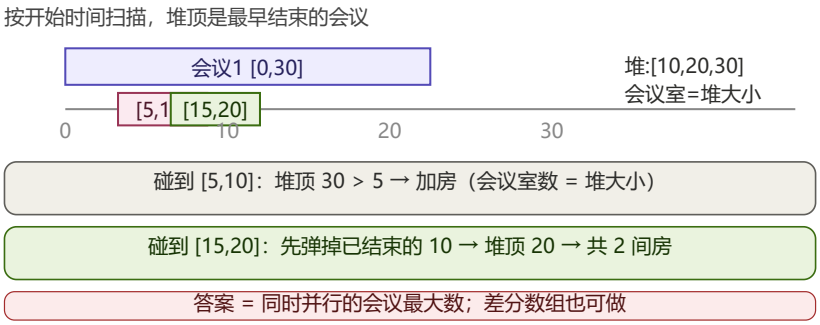


#253 会议室 II 中等 · 堆 / 贪心 / 数组（Premium）

给定会议时间区间，最少需要几间会议室。

```
[[0,30],[5,10],[15,20]]
→ 2
```

⚡ 最小堆按结束时间排：最早的结束还晚于当前开始就须加房。

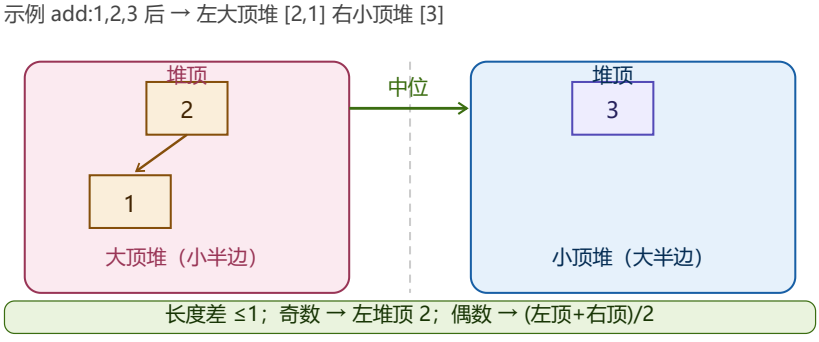


#295 数据流的中位数 困难 · 设计 / 堆 / 双指针

支持 addNum 与 findMedian 的数据结构。

```
addNum(1),addNum(2) → 1.5
addNum(3) → 2.0
```

⚡ 两个堆：大顶堆存较小一半、小顶堆存较大一半，堆顶即中位数。



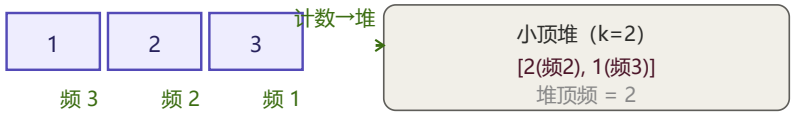
#347 前 K 个高频元素 中等 · 哈希表 / 堆 / 桶排序 / 快速选择

返回出现频率前 k 高的元素。

```
nums=[1,1,1,2,2,3], k=2 → [1,2]
```

⚡ 哈希计数 → 小顶堆维护 top-k；或桶按频率存值。

计数 {1:3, 2:2, 3:1}；小顶堆保留下标频率最高的 k 个



新元素频比堆顶大 → 弹出堆顶再入；堆内即 top-k 结果

桶排序：频次做桶下标，从高到低取够 k 个，O(n)

贪心算法

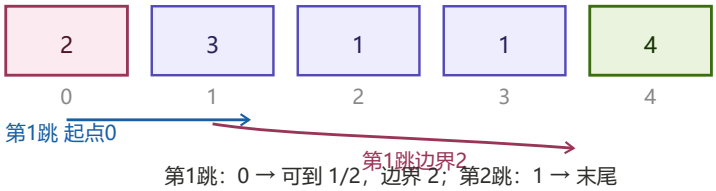
#45 跳跃游戏 II 中等 · 贪心 / 数组 / DP

从下标 0 跳到末尾，求最小跳跃次数。

```
nums=[2,3,1,1,4] → 2（跳 1 步到 1，再跳 3 步到 4）
```

⚡ BFS 式贪心：每跳划定「边界」，边界内探最大可达，出界即下一跳。

当前步 [l, r] 区间内，探出最远边界 next_r，步数+1



第1跳：0 → 可到 1/2，边界 2；第2跳：1 → 末尾

next_r = max(next_r, i+nums[i])；i 到边界时步数++，边界=next_r
O(n) 一维扫描，不需要 DP 表

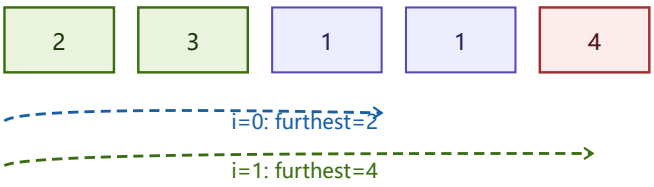
#55 跳跃游戏 中等 · 贪心 / 数组

向后跳跃最大 nums[i] 步，判断能否到终点。

```
nums = [2,3,1,1,4]
→ true
nums = [3,2,1,0,4]
→ false
```

⚡ 维护最远可达 furthest，i 超过 furthest 即失败；每步更新 furthest = max(furthest, i + nums[i])。

furthest 贪心扩张可及范围



i 每一步都 ≤ furthest 才能继续；furthest=4≥末位 → true

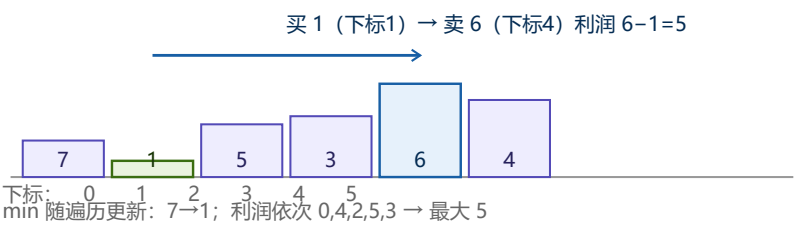
#121 买卖股票的最佳时机 简单 · 数组、动态规划

一次买入一次卖出，求最大利润（先买后卖）。

```
输入：[7,1,5,3,6,4]
输出：5
解释：第 2 天买 1，第 5 天卖 6，利润 5
```

⚡ 一边遍历一边更新最低价 min，当天价 − min 就是潜在利润，取最大。

记录到当前为止的最低价，算每个价位的潜在利润



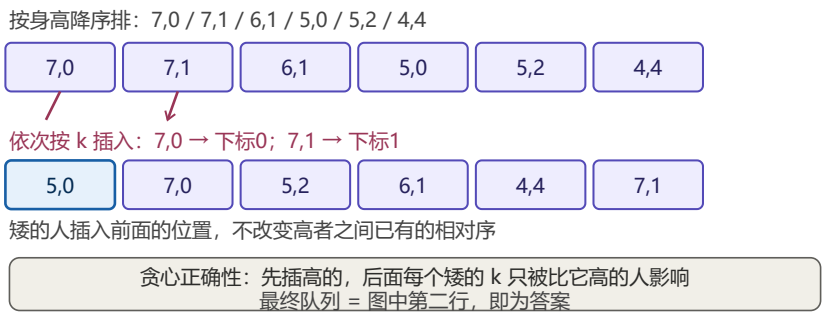
min 随遍历更新：7→1；利润依次 0,4,2,5,3 → 最大 5

#406 根据身高重建队列 中等 · 排序、贪心

按 people[i]=[身高 hi, 前面 ≥hi 的人数 ki] 重建正确队列。

```
输入：people = [[7,0],[4,4],[7,1],[5,0],[6,1],[5,2]]
输出：[[5,0],[7,0],[5,2],[6,1],[4,4],[7,1]]
```

⚡ 身高降序、同高 k 升序排序后，按 k 依次插入：k 就是插入下标，矮的后插不影响高的。

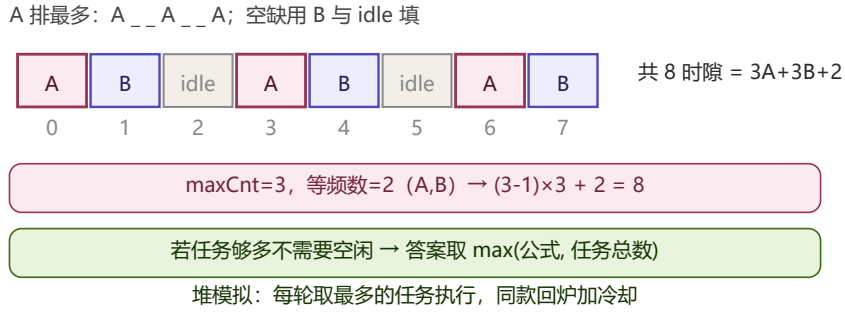


#621 任务调度器 中等 · 贪心 / 数组 / 哈希表 / 计数 / 堆

同类任务间隔至少 n，求完成全部任务的最短时间。

```
tasks=[A,A,A,B,B,B], n=2 → 8
```

⚡ 桶公式：(maxCnt-1)×(n+1) + 与 maxCnt 等频的个数；并与 len 取 max。

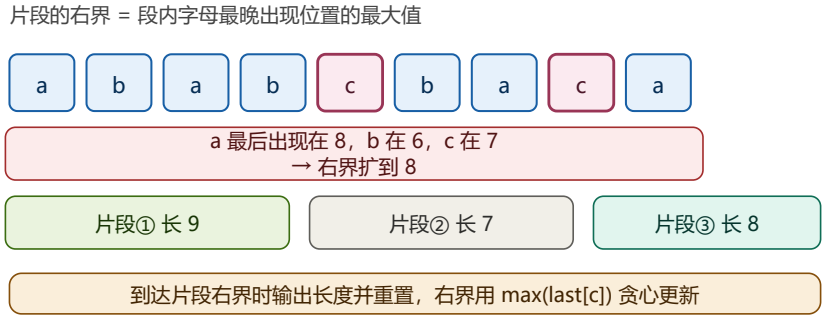


#763 划分字母区间 中等 · 贪心 / 哈希表 / 双指针

把字符串切成尽量多片段，同一字母只出现在一个片段中。

```
s = "ababcbacadefegdehijhklij"
→ [9, 7, 8]
片段 "ababcbaca" "defegde" "hijhklij"
```

⚡ 贪心：片段右界 = max(段内字母最后出现位置)，i 越过右界即切一刀。



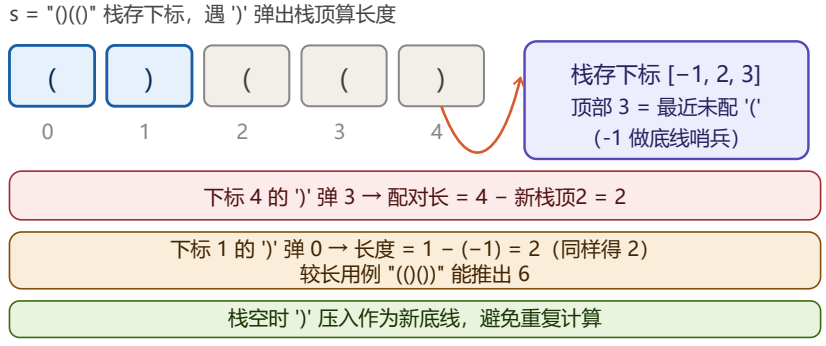
动态规划

#32 最长有效括号 困难 · 栈、字符串、动态规划

在只含 '(' 和 ')' 的串中，找最长的有效括号子串长度。

```
输入：s = "(()"
输出：2
最长有效括号子串是 "()"
```

⚡ 栈存下标：先压 -1 做底，'(' 入栈，')' 弹栈并算「当前 − 栈顶」更新最大长度。



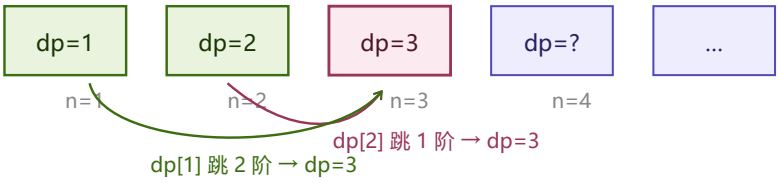
#70 爬楼梯 简单 · 动态规划 / 记忆化

每次 1 阶或 2 阶，爬到 n 阶共有几种方法。

```
n = 2 → 2
n = 3 → 3
n = 4 → 5
```

⚡ 斐波那契递推：dp[n] = dp[n-1] + dp[n-2]，滚动两变量即可，无需数组。

每一阶只可能从「前一阶」或「前两阶」跳上来



dp[n]=dp[n-1]+dp[n-2] → 1,2,3,5,8... 用 a,b 滚动

#96 不同的二叉搜索树 中等 · 树、BST、动态规划

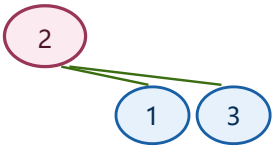
由 1..n 的节点能组成多少棵不同的 BST。

```
输入：n = 3
输出：5
```

⚡ 卡特兰数：dp[n] = Σ dp[k] · dp[n−1−k]，以每个数作根，左右子树分开计数。

dp[0]=1，dp[1]=1；n=3：枚举根 1,2,3

根=2（左 1 个，右 1 个）



dp[2] = dp[0]·dp[1]
+ dp[1]·dp[0] = 2

dp[3] = 1·2 + 1·1 + 2·1 = 5 ✓

公式：dp[i] = Σ_{k=0..i−1} dp[k] · dp[i−1−k]（k 为左子树个数）

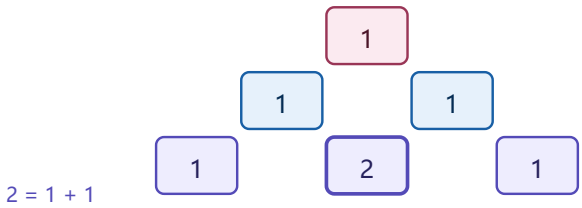
#118 杨辉三角 简单 · 数组、动态规划

生成杨辉三角（帕斯卡三角）前 numRows 行。

```
输入：numRows = 5
输出：[[1],[1,1],[1,2,1],[1,3,3,1],
[1,4,6,4,1]]
```

⚡ 每行首尾为 1，中间项 = 上一行相邻两项之和。

row[j] = row[i−1][j−1] + row[i−1][j]，首尾恒为 1



第4行 1 3 3 1；第5行 1 4 6 4 1
每行滚动更新：row[j] = 上一行 row[j−1] + row[j]

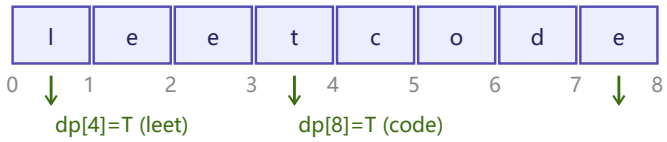
#139 单词拆分 中等 · 字典树 / 记忆化 / 哈希表 / 字符串 / DP

s 能否由字典中的单词拼接而成。

```
s="leetcode", dict=["leet","code"]
→ true
```

⚡ dp[i]=前 i 个字符可拼；j 枚举切点，dp[j] 且 s[j:i] 在字典则 dp[i]=true。

dp[0]=T；每个 i 试每个 j：dp[j]且s[j:i]∈dict→dp[i]=T



dp[j] 为 T 且在 dict 中 → dp[i]=T，否则试下一个 j

也可 DFS+记忆化：dfs(i) 从 i 向后试匹配字典词，命中进下一段

#152 乘积最大子数组 中等 · 数组、动态规划

找乘积最大的连续子数组，返回其乘积。

```
输入：nums = [2,3,-2,4]
输出：6
解释：子数组 [2,3] 有最大乘积 6
```

⚡ 双状态：同时维护「到当前位置的最大积 max 和最小积 min」，负数会翻转大小。

nums = [2, 3, -2, 4]；负号会让 max/min 互换



max: 2 → 6 → -2 → 4　　min: 2 → 3 → -12 → -48

遇 -2（交换法）：先交换 max/min，再 max=3×(-2)= -2，min=3×(-2)= -12
遇 4：max = max(4, -2×4) = 4，min = min(4, -12×4) = -48

遇到负数时先交换 max/min 再乘，即可同时处理符号
全局答案 = max 6（来自 [2,3]）

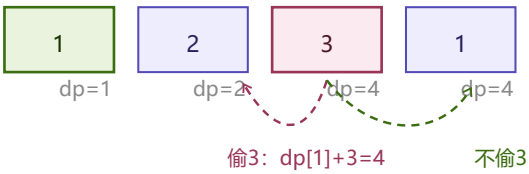
#198 打家劫舍 中等 · 动态规划 / 数组

相邻房屋不能同晚偷，求偷到的最大金额。

```
nums = [1,2,3,1]
→ 4（1 号 + 3 号）
nums = [2,7,9,3,1]
→ 12（2+9+1）
```

⚡ dp[i] = max(dp[i-1], dp[i-2] + nums[i])：不偷 i / 偷 i，取较大。

每个房子：要么偷（隔一家），要么不偷



偷3：dp[1]+3=4　　不偷3：dp[2]=4

dp[i]=max(dp[i-1], dp[i-2]+nums[i])；滚动 a,b 两变量即可

#279 完全平方数 中等 · BFS、数学、动态规划

把 n 拆成若干个完全平方数之和，求最少数量。

```
输入：n = 12
输出：3
解释：12 = 4 + 4 + 4
```

⚡ 动态规划：dp[i] = min(dp[i − j²]) + 1，完全平方数为物品的完全背包。

dp[12]：从 {1,4,9} 中选平方数凑 12 的最少数量



dp[12] 遍历 j = 1,2,3

12−1=11 → dp[11]=3；12−4=8 → dp[8]=2；12−9=3 → dp[3]=3

dp[12] = min(dp[11], dp[8], dp[3]) + 1 = 2 + 1 = 3 ✓

套路：正数 DP「最少步数」→ 完全背包/四平方定理

#300 最长递增子序列 中等 · 数组、二分、动态规划

找最长严格递增子序列的长度（可不连续）。

```
输入：nums = [10,9,2,5,3,7,101,18]
输出：4
解释：[2,3,7,101] 长度 4
```

⚡ O(n²) DP 或 O(nlogn) 贪心：维护 tails 递增数组，二分替换第一个 ≥ 当前值的元素。

用 tails[] 存「长度为 i 的 LIS 最小末尾」，整体严格递增



tails 变化：10 → 9 → 2 → [2,5] → [2,3] → [2,3,7] → [2,3,7,101]
新值若大于 tails 末尾 → 追加；否则二分替换第一个 ≥ 它的位置（贪心留小）

最终长度 = len(tails) = 4
严格递增：用 lower_bound 替换；非严格用 upper_bound（易错点）

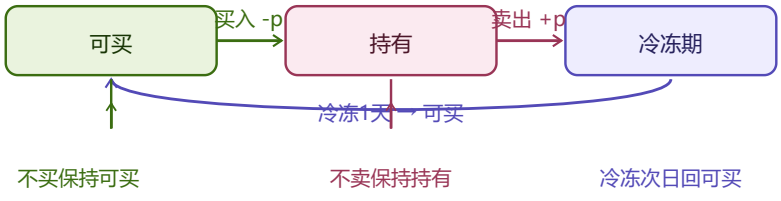
#309 买卖股票的最佳时机含冷冻期 中等 · DP

卖出的次日为冷冻期不能买入，求最大收益。

```
prices=[1,2,3,0,2] → 3
（买→卖→冷冻→买→卖）
```

⚡ 三态机：持有 / 不持有(可买) / 冷冻期，每天只跟相邻态转移。

三状态 DP：只有「可买」才能买入，「卖出」次日进冷冻



不买保持可买　　不卖保持持有　　冷冻次日回可买

答案 = max(可买, 冷冻) 即不持股的最大值；O(n)

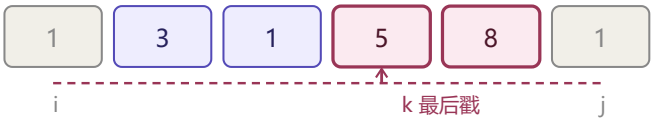
#312 戳气球 困难 · 数组、区间 DP

n 个气球各标一个数字，戳破气球 i 得 nums[左邻居]×nums[i]×nums[右邻居]（戳后邻居合并），求最大得分。

```
输入：nums = [3,1,5,8]
输出：167
3·1·5 + 3·5·8 + 1·3·8 + 1·8·1 = 167
```

⚡ 反向思维：枚举「最后一个戳的气球 k」，左右子区间 (i,k)、(k,j) 互相独立 → 区间 DP 按长度从小到大填表。

补哨兵 [1,3,1,5,8,1]；dp[i][j] = 开区间 (i,j) 全部戳完的最大得分



dp[i][k] + nums[i]·nums[k]·nums[j] + dp[k][j]

枚举 k：dp[i][j] = max(dp[i][j], 上式)

#322 零钱兑换 中等 · BFS、数组、动态规划、背包

用 coins 面额凑成 amount，返回最少硬币数，凑不出返回 -1。

```
输入：coins = [1, 2, 5], amount = 11
输出：3
解释：11 = 5 + 5 + 1
```

⚡ dp[j] = min(dp[j − coin]) + 1；完全背包按 amount 升序，每枚无限用。

coins = {1,2,5}，amount = 11；dp 从 0 推到 11



下标: 0 1 2 3 4 5 6 7 8 9 10 11

dp: 0 1 1 2 2 1 2 2 3 3 2 3

dp[11] 遍历: 11−5 → dp[6]=2 → 候选3；11−2... 取最小 3

每一步 dp 由左到右刷新，硬币无限用即为完全背包

凑不出（无穷大）保持 dp 值，最终仍无穷大返回 -1

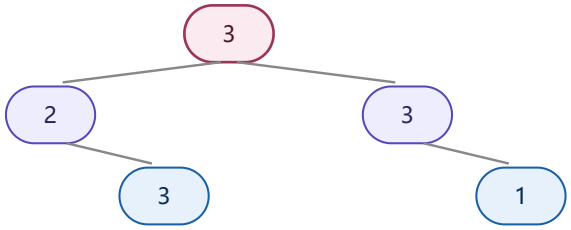
#337 打家劫舍 III 中等 · 树、DFS、树形 DP

二叉树小偷：不能偷相邻父子节点，求能偷的最大金额。

```
输入：root = [3,2,3,null,3,null,1]
输出：7
解释：3 + 3 + 1 = 7
```

⚡ 后序返回 (偷, 不偷) 两个值：偷 = 本值+子不偷；不偷 = 子取 max(偷,不偷)。

每个节点返回 (偷, 不偷)，父节点用子节点结果递推



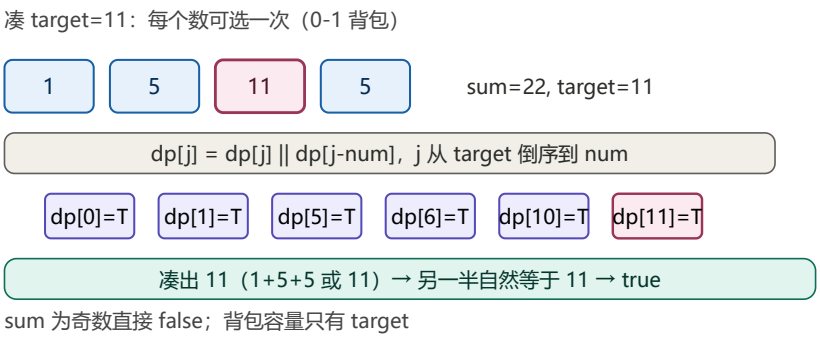
根偷 = 3 + 左子不偷(3) + 右子不偷(1) = 7
根不偷 = max(偷,不偷)左 + max(偷,不偷)右 = 3 + 3 = 6 → 答案 7

#416 分割等和子集 中等 · 0-1 背包 / 动态规划

能否把数组分成两个和相等的子集（即凑出总和一半）。

```
nums = [1,5,11,5]
→ true（[1,5,5] 与 [11]，各 11）
nums = [1,2,3,5]
→ false（和 11 为奇数）
```

⚡ 0-1 背包：target=sum/2，dp[j] 表示能否凑出 j；每个数只能取一次，倒序更新。



LeetCode 可视复习卡 • 背包DP

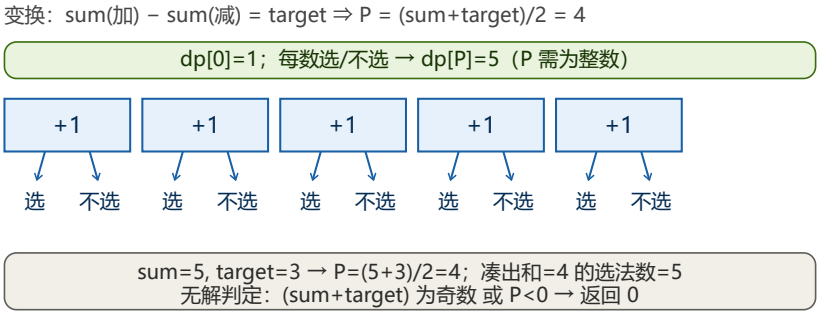
题面 + 示例 + 核心套路可视化 + 一句话口诀
(2026-09-26)

#494 目标和 中等 · 数组 / 动态规划 / 0-1 背包

给每个数加 + 或 −，求表达式的和恰好等于 target 的方法数。

```
nums = [1,1,1,1,1], target = 3
→ 输出：5
解释：共 5 种方法：
-1+1+1+1+1 = 3
+1-1+1+1+1 = 3
+1+1-1+1+1 = 3
+1+1+1-1+1 = 3
+1+1+1+1-1 = 3
```

⚡ 数学变换：设加号和为 P，则 2P = sum + target → 01 背包凑容量 P。



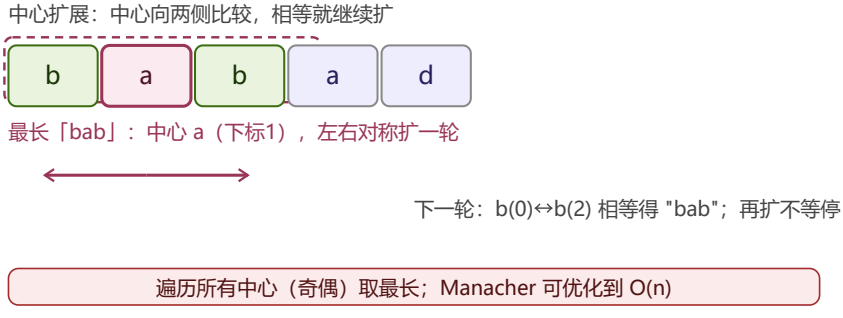
多维动态规划

#5 最长回文子串 中等 · 双指针、字符串、动态规划、Manacher

在字符串 s 中找到最长回文子串（正读反读相同）。

```
输入：s = "babad"
输出："bab"
解释："aba" 同样符合
```

⚡ 中心扩展：以每个中心向两侧比较，左右对称相等就继续扩，遍历全部中心取最长。



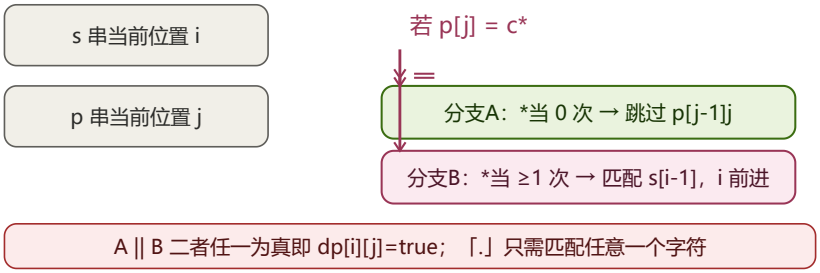
#10 正则表达式匹配 困难 · 动态规划 / 递归 / 字符串

支持「.」任意单字符与「*」前字符重复 0 次或多次，判断 p 能否匹配 s。

```
s = "aa", p = "a"
→ false
s = "aa", p = "a*"
→ true（* 让 a 重复 2 次，恰好匹配 "aa"）
```

⚡ 二维 DP：dp[i][j] 表示 s 前 i 个 vs p 前 j 个；遇 x* 分「*当 0 次」与「*当 ≥1 次」双分支。

核心：dp[i][j]，p[j] 为「字母+*」时两个分支



A || B 二者任一为真即 dp[i][j]=true；「.」只需匹配任意一个字符

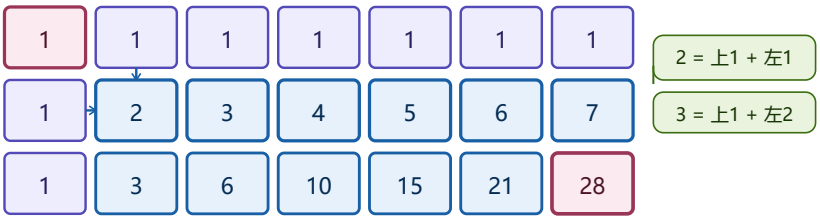
#62 不同路径 中等 · 数学、动态规划、组合数学

机器人从左上角走到右下角，每次只能向下或向右，求不同路径数。

```
输入：m = 3, n = 7
输出：28
```

⚡ 递推：dp[i][j] = dp[i−1][j] + dp[i][j−1]；首行首列都只有 1 条路。

3×7 网格每个格子 = 上方来 + 左方来；首行首列恒为 1



递推到底 → 终点 dp[2][6] = 28 ✓

#64 最小路径和 中等 · 数组、动态规划、矩阵

从左上走到右下，路径数字总和最小（只能右/下走）。

```
输入：grid = [[1,3,1],[1,5,1],[4,2,1]]
输出：7
解释：路径 1→3→1→1→1 和最小
```

⚡ 表格 DP：dp[i][j] = grid[i][j] + min(上, 左)；首行首列只累加一步方向。

表格 DP：dp[i][j] = grid[i][j] + min(上, 左)



dp 结果 = 每格取「上/左较小 + 自身」
→ 右下角 dp = 7（最短路径和）

沿 dp 最小路径反推：1→3→1→1→1，总 = 7

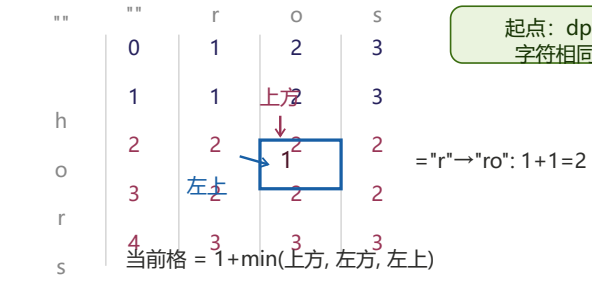
#72 编辑距离 中等 · 字符串 / DP

把 word1 转成 word2 的最少编辑次数（增/删/改各 1 次）。

```
word1="horse", word2="ros" → 3
```

⚡ dp[i][j]：从左、上、左上三个方向取最小，字符相同取左上。

dp[i][j] = min(上+1 删, 左+1 增, 左上+0/1 改)



起点：dp[0][j]=j, dp[i][0]=i
字符相同：取左上值不 +1

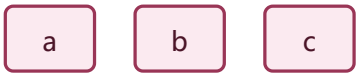
#647 回文子串 中等 · 双指针、字符串、动态规划

统计字符串中回文子串的个数。

```
输入：s = "abc"
输出：3
解释：三个回文子串："a","b","c"
```

⚡ 中心扩展：每个字符作中心向两边扩（奇数中心），每对相邻字符作偶数中心，计数。

s = "abc"：每个位置作中心向外扩展，左右相同即计数



奇数中心：a|b|c → 3 个　　偶数中心：ab,bc 左右不同 → 0 个

总计 3 + 0 = 3 ✓

扩展：l-- r++，只要 s[l]==s[r] 就继续计数
复杂度 O(n²)；dp[i][j] 递推亦可

#1143 最长公共子序列 中等 · 字符串、动态规划

返回两个字符串的最长公共子序列长度。

```
输入：text1 = "abcde", text2 = "ace"
输出：3
解释：最长公共子序列是 "ace"
```

⚡ 二维 DP：字符相等 dp[i][j]=dp[i−1][j−1]+1；否则取左/上较大值。

dp[i][j]：text1 前 i 与 text2 前 j 的 LCS 长度

	ε	a	c	e
a	1	1	1	
b	1	2	2	

行头：text2 = a c e

列头：text1 = a b c d e

dp[2][2]=2 = "ac"

右下角 dp[5][3]=3 ✓ 匹配沿对角线 +1，不匹配取 max(左,上)

技巧

#31 下一个排列 中等 · 数组 / 双指针

原地找字典序下一个更大的排列。

```
nums=[1,2,3] → [1,3,2]；[3,2,1] → [1,2,3]
```

⚡ 从右找第一个「升序拐点」，再从右找比它大的数交换，拐点后反转。

从右往左找第一个 a[i] < a[i+1] 的拐点



拐点 i=1 (2<3)

右侧比 2 大的最小数是 3 → 交换 i 与 j

交换后：



1 3 2 = 最小增幅

i 右侧是降序 → 反转成升序，得到最小增幅

整段降序则直接反转整数组（回到最小排列）

#75 颜色分类 中等 · 双指针 / 排序

三色（0/1/2）原地排序成 [0…0,1…1,2…2]。

```
nums = [2,0,2,1,1,0]
→ [0,0,1,1,2,2]
```

⚡ 三指针：p0 指向第一个 1，p2 指向第一个 2 前的位置；扫到 0 与 p0 换，扫到 2 与 p2 换。

扫描指针 i 与 p0/p2 交换，把 0 赶左、2 赶右



遇到 0 → 与 p0 交换，p0++，i++
遇到 2 → 与 p2 交换，p2--（i 不动，再查换回值）



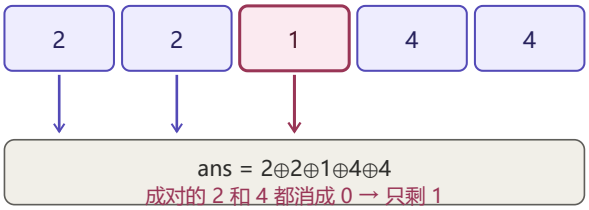
#136 只出现一次的数字 简单 · 位运算、数组

数组中除一个元素外都出现两次，找出只出现一次的元素。

```
输入：nums = [2,2,1]
输出：1
```

⚡ 异或：相同数异或得 0，0 与任何数异或得本身；全体异或一次即可。

从左到右累计异或所有数



ans = 2⊕2⊕1⊕4⊕4
成对的 2 和 4 都消成 0 → 只剩 1

性质：a⊕a=0，0⊕a=a，满足交换结合 → 唯一落单者留下

#169 多数元素 简单 · 数组、哈希表、摩尔投票

找到出现次数大于 ⌊n/2⌋ 的多数元素。

```
输入：nums = [3,2,3]
输出：3
```

⚡ 摩尔投票：候选 + 计数，相同 +1 不同 -1，减到 0 换候选；最后剩下的就是多数。

两两抵消：多数元素最后一定"剩"下来



3 → cnt=1　　遇 2 → cnt=0再读 3 → cnt=1

3 与 2 相消（计数归 0），第三个 3 当选候选 → cnt=1 ✓

原理：多数元素数量过半，抵消后必剩它
O(n) 时间 O(1) 空间

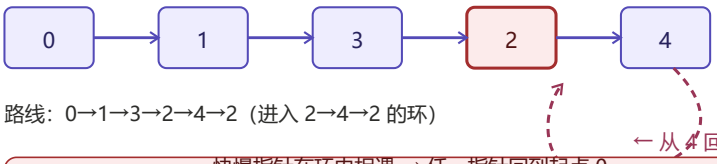
#287 寻找重复数 中等 · Floyd 判圈 / 数组 / 二分

n+1 个数取值范围 [1,n]，必有重复，找出它（不能修改数组）。

```
nums=[1,3,4,2,2] → 2
nums=[3,1,3,4,2] → 3
```

⚡ 把 i→nums[i] 看成「链表 next」：重复数就是环入口；快慢指针先相遇、再从头同步找入口。

节点=下标，箭头下一站=nums[下标] → 值 i 进了环



路线：0→1→3→2→4→2（进入 2→4→2 的环）

← 从 4 回到 2 成环

快慢指针在环内相遇 → 任一指针回到起点 0
两指针同速走，再相遇处即环入口（重复值 2）

不修改数组、不用额外哈希 → O(1) 空间判圈法

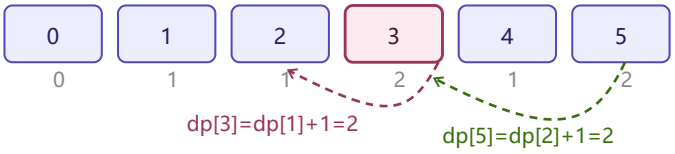
#338 比特位计数 简单 · 位运算 / 动态规划

对 0..n 每个数返回其二进制中 1 的个数。

```
n = 5
→ [0,1,1,2,1,2]
  （0=0,1=1
   2=10,3=11
   4=100,5=101）
```

⚡ dp[i] = dp[i >> 1] + (i & 1)：去掉最低位后的计数加上最低位本身。

i 的 1 个数 = i/2 的个数 + 最低位是否为 1



也可用「最低位 1」递推：dp[i]=dp[i & (i-1)]+1，去掉最后一个 1

#448 找到所有数组中消失的数字 简单 · 数组、哈希表

1..n 范围，找没在数组中出现的数字，O(1) 额外空间。

```
输入：nums = [4,3,2,7,8,2,3,1]
输出：[5,6]
```

⚡ 原地标记：值 v 出现过 → 把 nums[v−1] 标为负数；仍为正的下标 +1 即缺失数。

nums 值 v → 位置 v−1 置负号，表示 v 出现过



nums = [4,3,2,7,8,2,3,1]
依次把 4→标负位3... 遍历后：

位置 0..7 中，5、6 两处仍未变负 → 缺失 [5,6]

技巧：利用数组本身当哈希表，值域恰为下标范围 → O(1) 空间

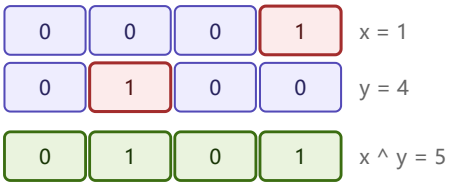
#461 汉明距离 简单 · 位运算

两个整数对应二进制位不同的个数。

```
输入：x = 1, y = 4
输出：2
解释：1 (0 0 0 1)，4 (0 1 0 0)，↑↑ 两处不同
```

⚡ 先异或得到所有不同位（结果里是 1 的位），再统计二进制中 1 的个数（bit_count）。

两数异或：不同位变 1 → 数 1 的个数即距离



1 ^ 4 = 5（0101），含 2 个 1 → 汉明距离 = 2
核心两步：① x ^ y 找不同位 ② bit_count 统计 1（或 n &= n-1）
0001 vs 0100：第 0、1 位不同 → 距离 2